Soluciones de almacenamiento de NetApp para Apache Spark

NetApp Solutions

NetApp
March 01, 2023

This PDF was generated from https://docs.netapp.com/es-es/netapp-solutions/data-analytics/apache-spark-solution-overview.html on March 01, 2023. Always check docs.netapp.com for the latest.

Tabla de Contenido

Soluciones de almacenamiento de NetApp para Apache Spark . . . 1
TR-4570: Soluciones de almacenamiento de NetApp para Apache Spark: Arquitectura, casos prácticos y resultados de rendimiento . . . 1
Público objetivo . . . 6
Tecnología de soluciones . . . 7
Información general de las soluciones Spark de NetApp . . . 9
Resumen de casos de uso . . . 11
Principales casos de uso y arquitecturas de IA, APRENDIZAJE AUTOMÁTICO y aprendizaje profundo . . . 14
Resultados de las pruebas . . . 19
Solución de cloud híbrido . . . 30
Scripts Python para cada caso de uso principal . . . 32
Conclusión . . . 50
Dónde encontrar información adicional . . . 50

Soluciones de almacenamiento de NetApp para Apache Spark

TR-4570: Soluciones de almacenamiento de NetApp para Apache Spark: Arquitectura, casos prácticos y resultados de rendimiento

Rick Huang, Karthikeyan Nagalingam, NetApp

Este documento se centra en la arquitectura Apache Spark, los casos prácticos de clientes y la cartera de almacenamiento de NetApp relacionada con el análisis de Big Data e inteligencia artificial (IA). También presenta diversos resultados de pruebas utilizando herramientas estándar del sector de IA, aprendizaje automático (ML, por sus siglas en inglés) y aprendizaje profundo (DL, por sus siglas en inglés) frente a un sistema Hadoop típico, de modo que pueda elegir la solución Spark adecuada. Para empezar, necesita una arquitectura Spark, componentes adecuados y dos modos de puesta en marcha (clúster y cliente).

Este documento también proporciona casos de uso para clientes que se ocupan de problemas relacionados con la configuración. En él se aborda una descripción general de la cartera de almacenamiento de NetApp relevante para análisis de Big Data e IA, ML y DL con Spark. Después, terminamos con los resultados de las pruebas obtenidas de los casos prácticos específicos de Spark y la cartera de soluciones de NetApp Spark.

Retos del cliente

Esta sección se centra en los retos de los clientes con análisis de Big Data e IA/ML/DL en sectores de crecimiento de datos como el comercio minorista, el marketing digital, la banca, la fabricación discreta, la fabricación de procesos, ee. uu. y servicios profesionales.

Rendimiento imprevisible

Las puestas en marcha tradicionales de Hadoop suelen usar hardware genérico. Para mejorar el rendimiento, debe ajustar la red, el sistema operativo, el clúster de Hadoop, los componentes del ecosistema como Spark y el hardware. Aunque ajuste cada capa, puede resultar difícil lograr los niveles de rendimiento deseados, porque Hadoop utiliza hardware genérico que no se ha diseñado para ofrecer un alto rendimiento en su entorno.

Fallos de medios y nodos

Incluso en condiciones normales, el hardware genérico es propenso a sufrir fallos. Si falla un disco en un nodo de datos, el maestro de Hadoop considera de forma predeterminada que ese nodo no está en buen estado. A continuación, copia datos específicos de ese nodo a través de la red de réplicas en un nodo en buen estado. Este proceso ralentiza los paquetes de red para cualquier trabajo de Hadoop. A continuación, el clúster debe volver a copiar los datos y eliminar los datos replicados en exceso cuando el nodo que no está en buen estado vuelva.

Bloqueo del proveedor de Hadoop

Los distribuidores de Hadoop tienen su propia distribución de Hadoop con su propio control de versiones, lo que bloquea al cliente para dichas distribuciones. Sin embargo, muchos clientes requieren compatibilidad con el análisis en memoria que no enlaza al cliente con distribuciones específicas de Hadoop. Necesitan tener libertad para cambiar las distribuciones y llevar sus análisis con ellas.

Falta de apoyo para más de un idioma

Los clientes a menudo requieren soporte para varios idiomas además de los programas Java de MapReduce para ejecutar sus trabajos. Las opciones como SQL y scripts proporcionan más flexibilidad para obtener respuestas, más opciones para organizar y recuperar datos y formas más rápidas de mover datos a un marco de análisis.

Dificultad de uso

Desde hace algún tiempo, la gente se ha quejado de que Hadoop es difícil de usar. Aunque Hadoop se ha vuelto más sencillo y potente con cada nueva versión, esta crítica se ha mantenido. Hadoop requiere que comprenda los patrones de programación de Java y MapReduce, un reto para los administradores de bases de datos y las personas con conjuntos de habilidades de scripting tradicionales.

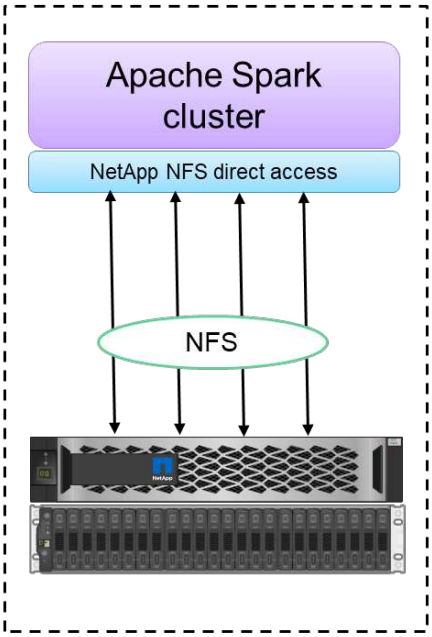
Marcos y herramientas complicados

Los equipos de IA de las empresas se enfrentan a diferentes retos. Incluso con conocimientos especializados en ciencia de datos, es posible que las herramientas y los marcos para diferentes ecosistemas y aplicaciones de puesta en marcha no se traduzcan simplemente de uno a otro. Una plataforma de ciencia de datos debe integrarse perfectamente con las plataformas de big data correspondientes basadas en Spark con facilidad de movimiento de datos, modelos reutilizables, código listo para usar y herramientas que admiten las mejores prácticas para la creación de prototipos, la validación, el control de versiones, el uso compartido, la reutilización, y poner en marcha rápidamente modelos para la producción.

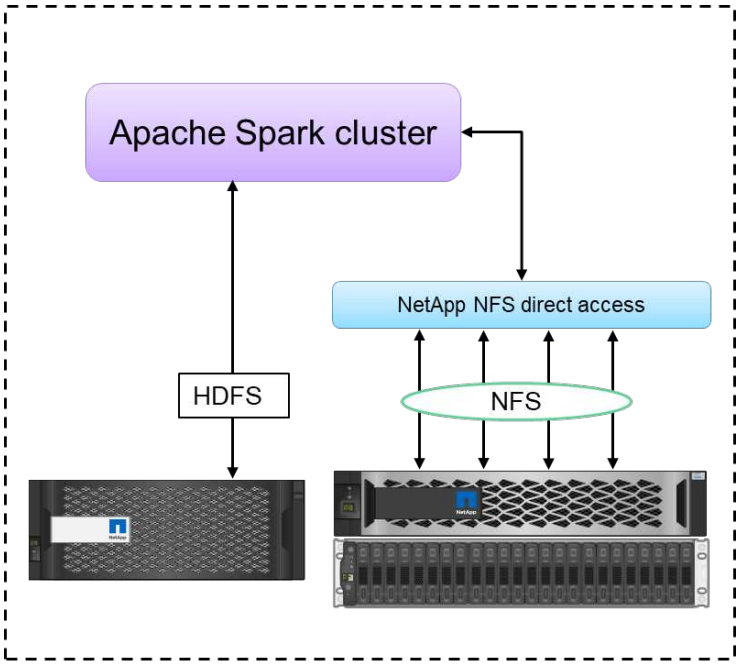
¿Por qué elegir NetApp?

NetApp puede mejorar su experiencia con Spark de las siguientes maneras:

- El acceso directo de NFS de NetApp (que se muestra en la siguiente figura) permite a los clientes ejecutar trabajos de análisis de Big Data en sus datos NFSv3 o NFSv4 existentes o nuevos sin necesidad de mover ni copiar los datos. Evita múltiples copias de datos y elimina la necesidad de sincronizar los datos con un origen.
- Almacenamiento más eficiente y menos replicación de servidor. Por ejemplo, la solución E-Series para Hadoop de NetApp requiere dos en lugar de tres réplicas de los datos y la solución para Hadoop de FAS requiere un origen de datos pero no una replicación o copias de estos. Las soluciones de almacenamiento de NetApp también producen menos tráfico de servidor a servidor.
- Mejor comportamiento del clúster y de las tareas de Hadoop durante el fallo de unidades y nodos.
- Mejor rendimiento de procesamiento de datos.



Configuration 1: NFS as primary storage



Configuration 2: HDFS and NFS in single Spark cluster

Por ejemplo, en el sector financiero y sanitario, el traslado de datos de un lugar a otro debe cumplir con obligaciones legales, lo que no es una tarea fácil. En esta situación, el acceso directo NFS de NetApp analiza los datos financieros y sanitarios desde su ubicación original. Otra ventaja clave es que el uso del acceso directo de NFS de NetApp simplifica la protección de datos de Hadoop mediante comandos nativos de Hadoop y permite flujos de trabajo de protección de datos con la cartera de gestión de datos enriquecidos de NetApp.

El acceso directo de NFS de NetApp proporciona dos tipos de opciones de puesta en marcha para clústeres Hadoop/Spark:

- De forma predeterminada, los clústeres de Hadoop o Spark utilizan el sistema de archivos distribuidos de Hadoop (HDFS) para el almacenamiento de datos y el sistema de archivos predeterminado. El acceso directo de NFS de NetApp puede sustituir el HDFS predeterminado por el almacenamiento NFS como sistema de archivos predeterminado, lo que permite el análisis directo de datos NFS.
- En otra opción de puesta en marcha, el acceso directo de NFS de NetApp admite la configuración de NFS como almacenamiento adicional junto con HDFS en un único clúster de Hadoop o Spark. En este caso, el cliente puede compartir datos a través de las exportaciones NFS y acceder a ellos desde el mismo clúster junto con los datos de HDFS.

Algunas de las ventajas clave del acceso directo de NFS de NetApp son las siguientes:

- Análisis de los datos desde su ubicación actual, lo que evita la tarea que requiere tiempo y rendimiento, consistente en mover datos de análisis a una infraestructura de Hadoop como HDFS.
- Reducción del número de réplicas de tres a una.
- Permitiendo a los usuarios desacoplar los recursos informáticos y del almacenamiento para escalarlos de forma independiente.
- Protege los datos empresariales aprovechando las funcionalidades de gestión de datos enriquecidos de ONTAP.
- Certificado en la plataforma de datos Hortonworks.
- Habilitar la puesta en marcha de análisis de datos híbridos.

- Reducción del tiempo de backup aprovechando la funcionalidad de multisubproceso dinámica.

Consulte "TR-4657: Soluciones de datos para el cloud híbrido de NetApp: Spark y Hadoop en función de casos prácticos de clientes" Para realizar backups de los datos de Hadoop, realice backups y recuperación ante desastres desde el cloud a las instalaciones, y permita DevTest en los datos de Hadoop existentes, la protección de datos y la conectividad multicloud, así como acelerar las cargas de trabajo analíticas.

En las siguientes secciones se describen las funcionalidades de almacenamiento que son importantes para los clientes de Spark.

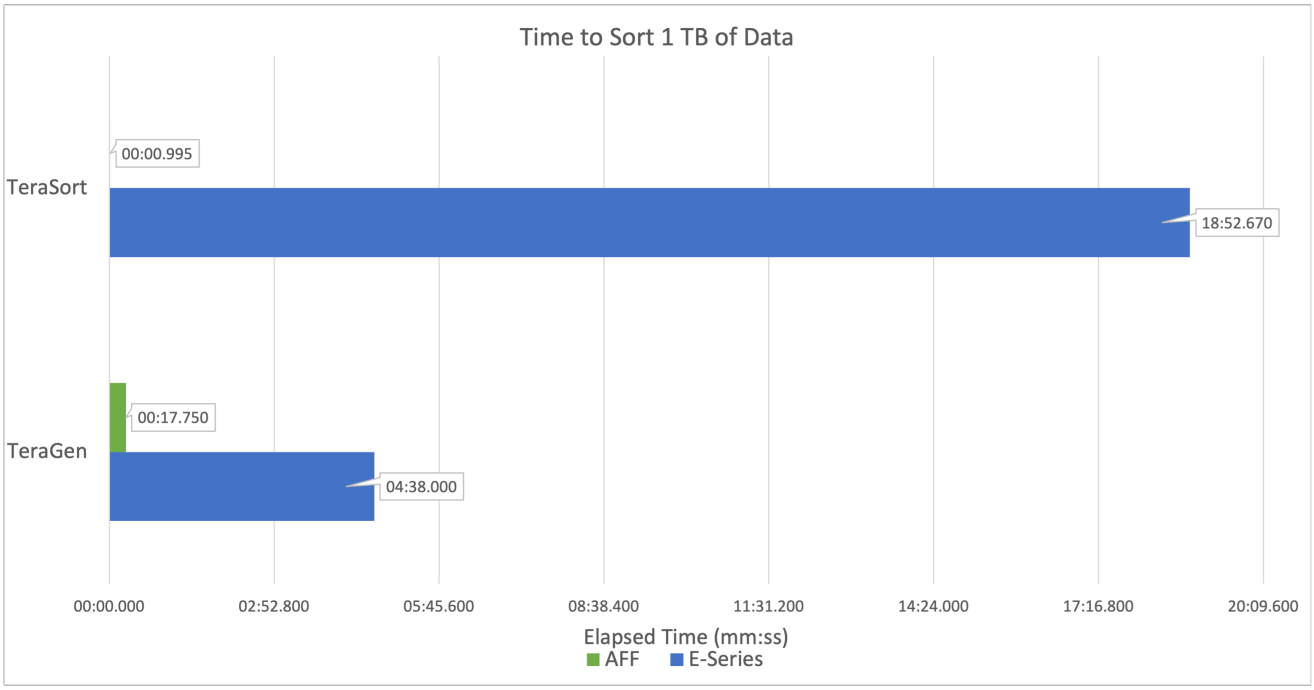
Niveles de almacenamiento

Con el almacenamiento por niveles de Hadoop, puede almacenar archivos con diferentes tipos de almacenamiento según una normativa de almacenamiento. Los tipos de almacenamiento incluyen `hot`, `cold`, `warm`, `all_ssd`, `one_ssd`, y. `lazy_persist`.

<<<<<<<<< HEAD validamos el almacenamiento por niveles de Hadoop en una controladora de almacenamiento AFF de NetApp y una controladora de almacenamiento E-Series con unidades SSD y SAS con políticas de almacenamiento diferentes. El clúster Spark con AFF-A800 tiene cuatro nodos de trabajo de computación, mientras que el clúster con E-Series tiene ocho. Esto se compara principalmente con el rendimiento de las unidades de estado sólido (SSD) en comparación con los discos de disco duro (HDD).

Validamos el almacenamiento por niveles de Hadoop en una controladora de almacenamiento AFF de NetApp y una controladora de almacenamiento E-Series con unidades SSD y SAS con políticas de almacenamiento diferentes. El clúster Spark con AFF-A800 tiene cuatro nodos de trabajo de computación, mientras que el clúster con E-Series tiene ocho. Esto se realizó principalmente para comparar el rendimiento de las unidades de estado sólido con los discos duros. >>>>>> a51c9ddf73ca61120ce05edc7b0b9607b96eae

La siguiente figura muestra el rendimiento de las soluciones de NetApp para SSD de Hadoop.



- La configuración de NL-SAS de referencia utilizó ocho nodos de computación y 96 unidades NL-SAS. Esta configuración generó 1 TB de datos en 4 minutos y 38 segundos. Consulte "TR-3969 solución E-Series de NetApp para Hadoop" para obtener detalles sobre el clúster y la configuración de almacenamiento.
- Con TeraGen, la configuración de SSD generó 1 TB de datos 15,6 veces más rápido que la configuración NL-SAS. Además, la configuración de SSD utilizó la mitad de la cantidad de nodos de computación y la mitad de la cantidad de unidades de disco (24 unidades SSD en total). Según el tiempo para completar la tarea, era casi el doble de rápido que la configuración de NL-SAS.
- Con TeraSort, la configuración de SSD ordena 1 TB de datos 1138.36 veces más rápido que la configuración de NL-SAS. Además, la configuración de SSD utilizó la mitad de la cantidad de nodos de computación y la mitad de la cantidad de unidades de disco (24 unidades SSD en total). Por lo tanto, por unidad, era aproximadamente tres veces más rápida que la configuración de NL-SAS. <<<<<<<<< CABEZA
- La ventaja es que la transición de los discos giratorios a all-flash mejora el rendimiento. El número de nodos de computación no era el cuello de botella. Con el almacenamiento all-flash de NetApp, el rendimiento del tiempo de ejecución se escala bien.
- Con NFS, los datos eran funcionalmente equivalentes a los agrupados en pools, lo que puede reducir el número de nodos de computación en función de la carga de trabajo. Los usuarios del clúster de Apache Spark no tienen que reequilibrar manualmente los datos cuando se cambia el número de nodos de computación.

- En resumen, la transición de discos giratorios a all-flash mejora el rendimiento. El número de nodos de computación no era el cuello de botella. Con el almacenamiento all-flash de NetApp, el rendimiento del tiempo de ejecución se escala bien.
- Con NFS, los datos eran funcionalmente equivalentes a los agrupados en pools, lo que puede reducir el número de nodos de computación en función de la carga de trabajo. Los usuarios de clúster de Apache Spark no necesitan reequilibrar manualmente los datos cuando se cambia el número de nodos de computación. >>>>>> a51c9ddf73ca61120ce05edc7b0b9607b96eae

Escalado del rendimiento: Escalado horizontal

Cuando necesite más capacidad informática de un clúster de Hadoop en una solución AFF, puede añadir nodos de datos con un número adecuado de controladoras de almacenamiento. NetApp recomienda comenzar con cuatro nodos de datos por cabina de controladora de almacenamiento y aumentar el número a ocho nodos de datos por controladora de almacenamiento, en función de las características de la carga de trabajo.

AFF y FAS son perfectos para análisis in situ. En función de los requisitos de computación, puede agregar administradores de nodos y las operaciones no disruptivas le permiten agregar una controladora de almacenamiento bajo demanda sin tiempo de inactividad. Ofrecemos completas funciones con AFF y FAS, como soporte de medios NVME, eficiencia garantizada, reducción de datos, CALIDAD DE SERVICIO, análisis predictivos, organización en niveles del cloud, replicación, puesta en marcha de cloud y seguridad. Para ayudar a los clientes a cumplir sus requisitos, NetApp ofrece funciones como análisis del sistema de archivos, cuotas y equilibrio de carga integrado sin costes de licencia adicionales. NetApp ofrece un mejor rendimiento en el número de tareas simultáneas, menor latencia, operaciones más sencillas y un mayor rendimiento de gigabytes por segundo que el de la competencia. Además, Cloud Volumes ONTAP de NetApp se ejecuta en los tres principales proveedores de cloud.

Escalado del rendimiento: Escalado vertical

Las funciones de escalado vertical le permiten agregar unidades de disco a sistemas AFF, FAS y E-Series cuando necesita capacidad de almacenamiento adicional. Con Cloud Volumes ONTAP, el escalado del almacenamiento a nivel de PB es una combinación de dos factores: La organización en niveles de los datos que se usan con poca frecuencia en el almacenamiento de objetos desde el almacenamiento de bloques y la apilamiento de licencias Cloud Volumes ONTAP sin necesidad de procesamiento adicional.

Múltiples protocolos

Los sistemas NetApp son compatibles con la mayoría de protocolos para puestas en marcha de Hadoop, como SAS, iSCSI, FCP, InfiniBand, Y NFS.

Soluciones operativas y compatibles

Las soluciones para Hadoop que se describen en este documento son compatibles con NetApp. Estas soluciones también están certificadas con los principales distribuidores de Hadoop. Para obtener más información, consulte "MAPR" sitio, el "Hortonworks" Local y Cloudera "certificación" y.. "como partner" sitios web.

"Siguiente: Público objetivo."

Público objetivo

"Anterior: Descripción general de la solución."

El mundo del análisis y la ciencia de datos tiene múltiples disciplinas DE TECNOLOGÍA y negocio:

- El científico de datos necesita la flexibilidad para poder usar las herramientas y las bibliotecas de elección.
- El ingeniero de datos necesita saber cómo fluyen los datos y dónde residen.
- Un ingeniero de DevOps necesita las herramientas necesarias para integrar nuevas aplicaciones de IA y ML en sus canalizaciones de CI y de CD.
- Los administradores y arquitectos de cloud deben poder configurar y gestionar recursos de cloud híbrido.
- Los usuarios empresariales quieren tener acceso a aplicaciones de análisis, IA, APRENDIZAJE AUTOMÁTICO y aprendizaje profundo.

En este informe técnico, describimos cómo AFF, E-Series, StorageGRID, acceso directo NFS, Apache Spark, Horovod, y Keras ayudan a cada uno de estos papeles a aportar valor a las empresas.

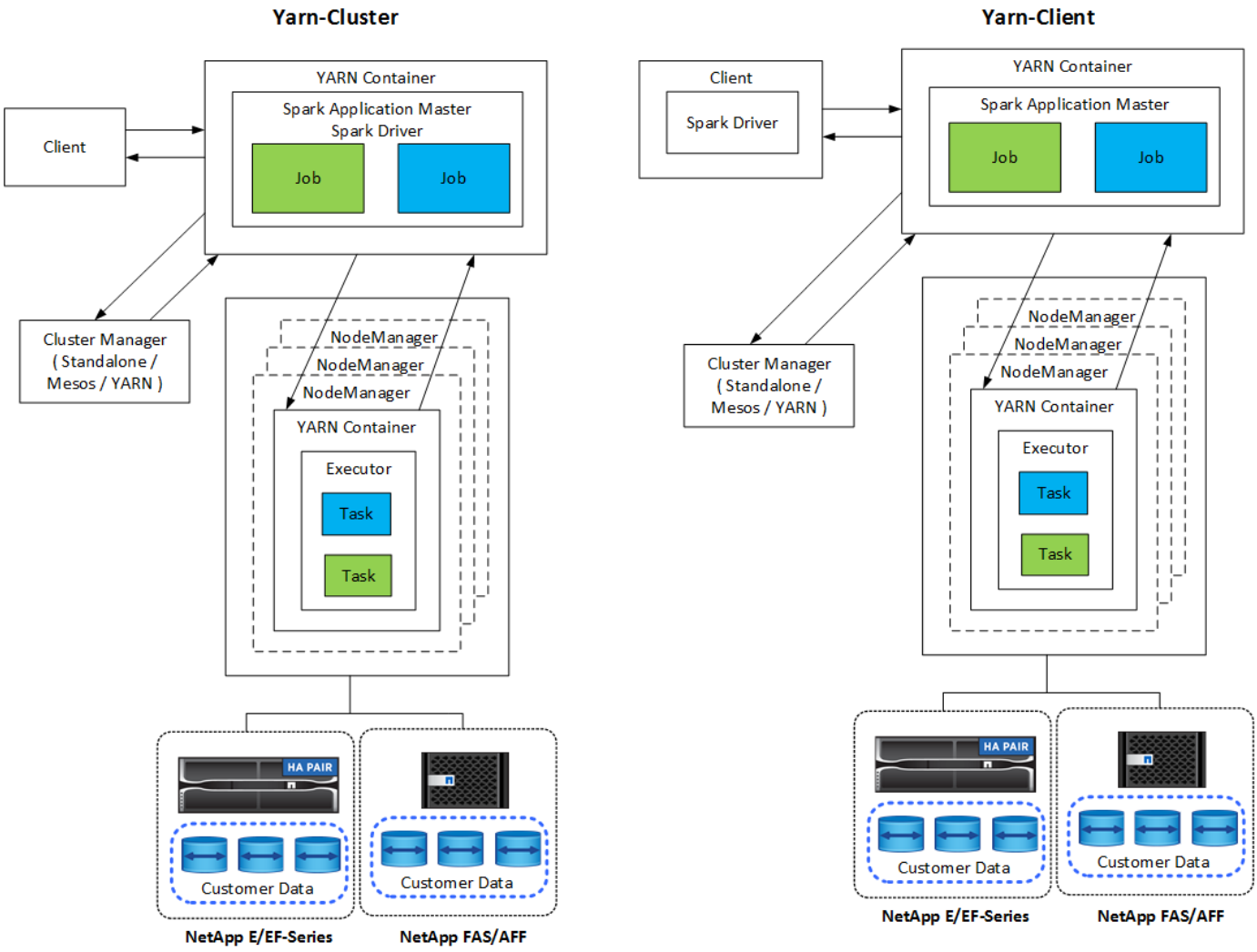
"Siguiente: Tecnología de soluciones."

Tecnología de soluciones

"Anterior: Público objetivo."

Apache Spark es un popular marco de programación para escribir aplicaciones de Hadoop que funciona directamente con el sistema de archivos distribuidos de Hadoop (HDFS). Spark está lista para la producción, admite el procesamiento de datos en streaming y es más rápida que MapReduce. Spark cuenta con un almacenamiento en caché de datos en memoria configurable para una iteración eficiente y la shell Spark es interactiva para aprender y explorar datos. Con Spark puede crear aplicaciones en Python, Scala o Java. Las aplicaciones SPARK consisten en uno o más trabajos que tienen una o más tareas.

Cada aplicación Spark tiene un controlador Spark. En el modo YARN-Client, el controlador se ejecuta en el cliente localmente. En el modo HILADO-Cluster, el controlador se ejecuta en el clúster en el maestro de aplicaciones. En el modo de clúster, la aplicación continúa ejecutándose incluso si el cliente se desconecta.



Hay tres administradores de clústeres:

- **Independiente.** este administrador forma parte de Spark, lo que facilita la configuración de un clúster.
- * Apache Mesos.* este es un gestor general de clusters que también ejecuta MapReduce y otras aplicaciones.
- **Hadoop YARN.** esto es un gestor de recursos en Hadoop 3.

El conjunto de datos distribuido resiliente (RDD) es el componente principal de Spark. RDD recrea los datos perdidos y que faltan de los datos almacenados en la memoria del clúster y almacena los datos iniciales que provienen de un archivo o que se crean mediante programación. Los RDD se crean a partir de archivos, datos en memoria u otro RDD. La programación SPARK realiza dos operaciones: Transformación y acciones. La transformación crea una RDD nueva basada en una existente. Las acciones devuelven un valor de un RDD.

Las transformaciones y acciones también se aplican a los conjuntos de datos de Spark y DataFrames. Un conjunto de datos es una colección distribuida de datos que proporciona las ventajas de los RDD (escritura sólida, uso de funciones lambda) con las ventajas del motor de ejecución optimizado de Spark SQL. Un conjunto de datos se puede construir a partir de objetos JVM y, a continuación, manipular mediante transformaciones funcionales (mapa, mapa plano, filtro, etc.). Un DataFrame es un conjunto de datos organizado en columnas con nombre. Es conceptualmente equivalente a una tabla de una base de datos relacional o un marco de datos de R/Python. Los DataFrames pueden construirse a partir de una amplia variedad de orígenes como archivos de datos estructurados, tablas en Hive/HBase, bases de datos externas en las instalaciones o en el cloud, o RDDs existentes.

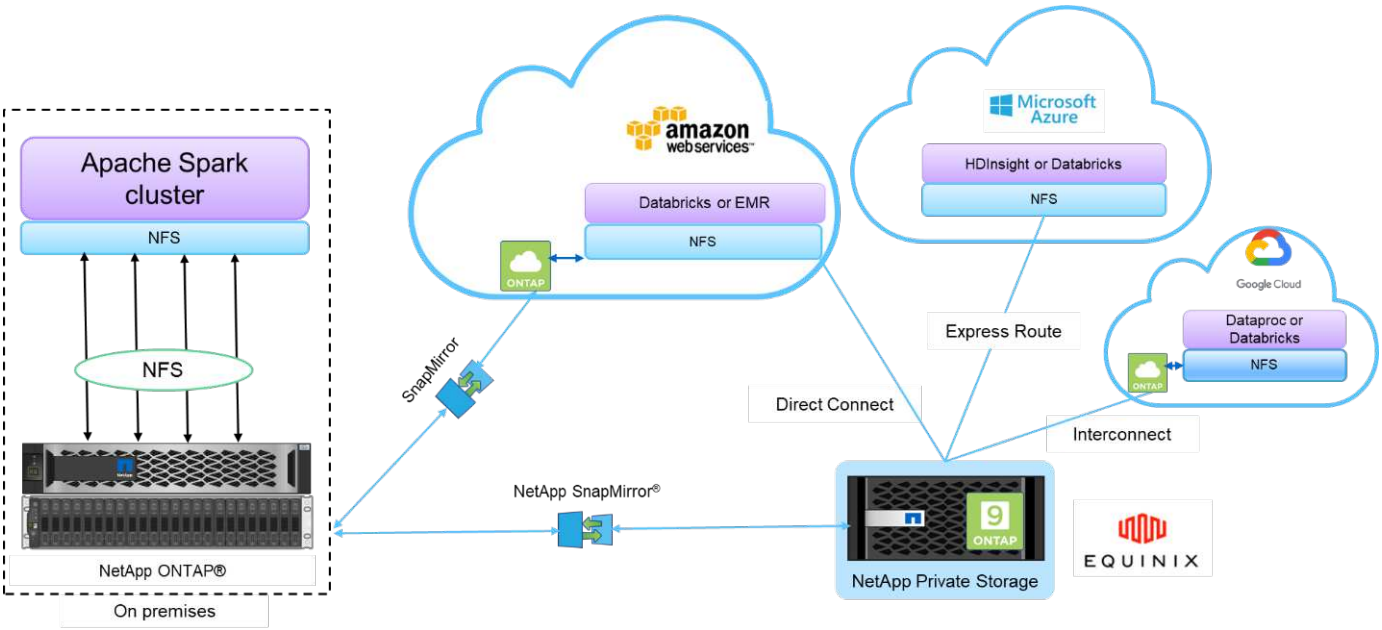
Las aplicaciones SPARK incluyen uno o más trabajos de Spark. Los trabajos ejecutan tareas en los ejecutores y los ejecutores se ejecutan en contenedores DE HILADOS. Cada ejecutor se ejecuta en un único contenedor, y existen ejecutores a lo largo de la vida de una aplicación. Un ejecutor se fija después de que se inicia la aplicación, Y EL HILADO no cambia el tamaño del contenedor ya asignado. Un ejecutor puede ejecutar tareas de forma simultánea en los datos de la memoria.

"Siguiente: Información general de las soluciones Spark de NetApp."

Información general de las soluciones Spark de NetApp

"Anterior: Tecnología de soluciones."

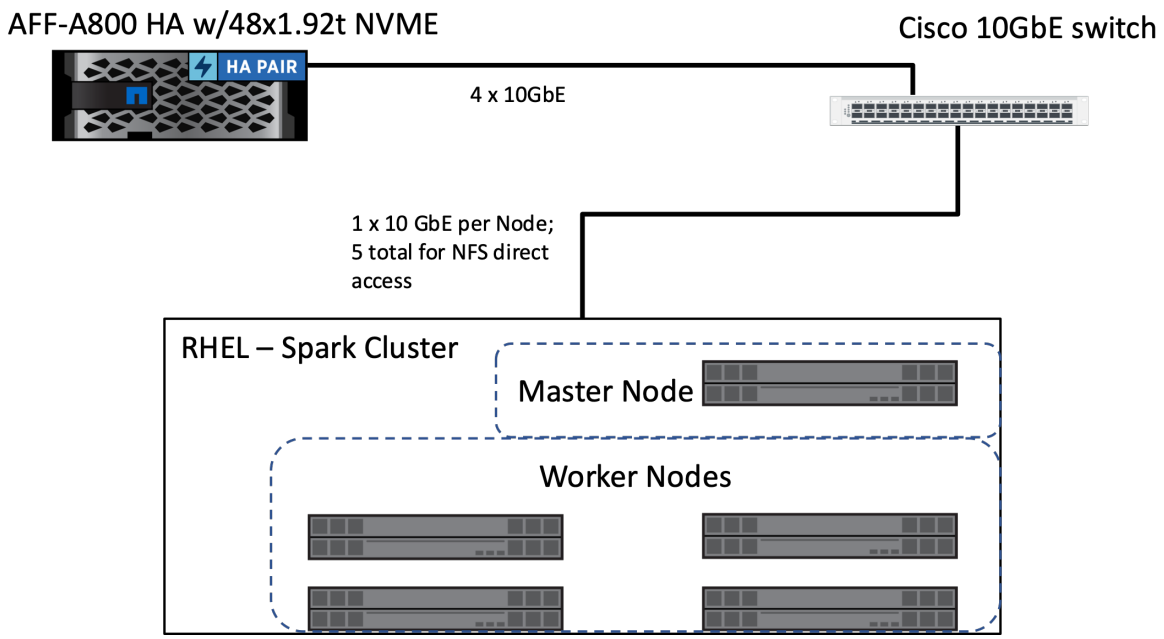
NetApp cuenta con tres carteras de almacenamiento: FAS/AFF, E-Series y Cloud Volumes ONTAP. Hemos validado AFF y E-Series con el sistema de almacenamiento ONTAP para soluciones Hadoop con Apache Spark. El Data Fabric con tecnología de NetApp integra servicios y aplicaciones de gestión de datos (elementos básicos) para el acceso, control, protección y seguridad de los datos, como se muestra en la siguiente figura.



Los elementos básicos de la figura anterior incluyen:

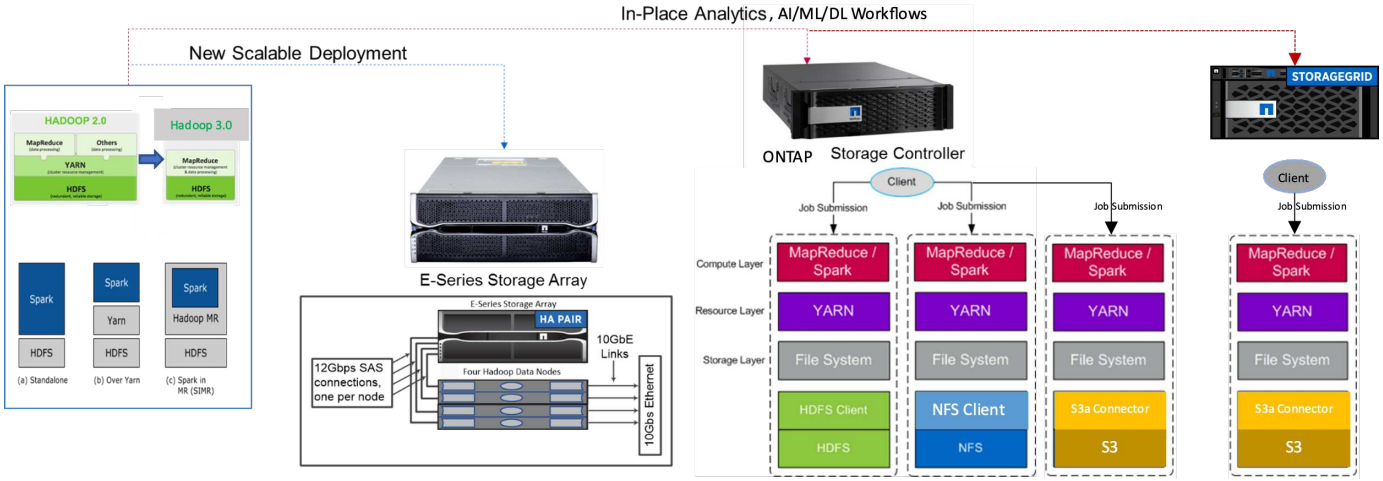
- **Acceso directo NFS de NetApp.** proporciona los últimos clústeres Hadoop y Spark con acceso directo a volúmenes NFS de NetApp sin requisitos adicionales de software o controlador.
- **Servicios de volúmenes de cloud y Cloud Volumes ONTAP de NetApp.** almacenamiento conectado definido por software basado en ONTAP que se ejecuta en Amazon Web Services (AWS) o Azure NetApp Files (ANF) en servicios de cloud de Microsoft Azure.
- **La tecnología SnapMirror de NetApp.** ofrece funcionalidades de protección de datos entre instancias locales y de cloud de ONTAP o NPS.
- **Proveedores de servicios en la nube.** estos proveedores incluyen AWS, Microsoft Azure, Google Cloud e IBM Cloud.
- **PaaS.** Servicios de análisis basados en cloud como Amazon Elastic MapReduce (EMR) y Databricks en AWS, así como Microsoft Azure HDInsight y Azure Databricks.

En la siguiente figura se muestra la solución Spark con el almacenamiento de NetApp.

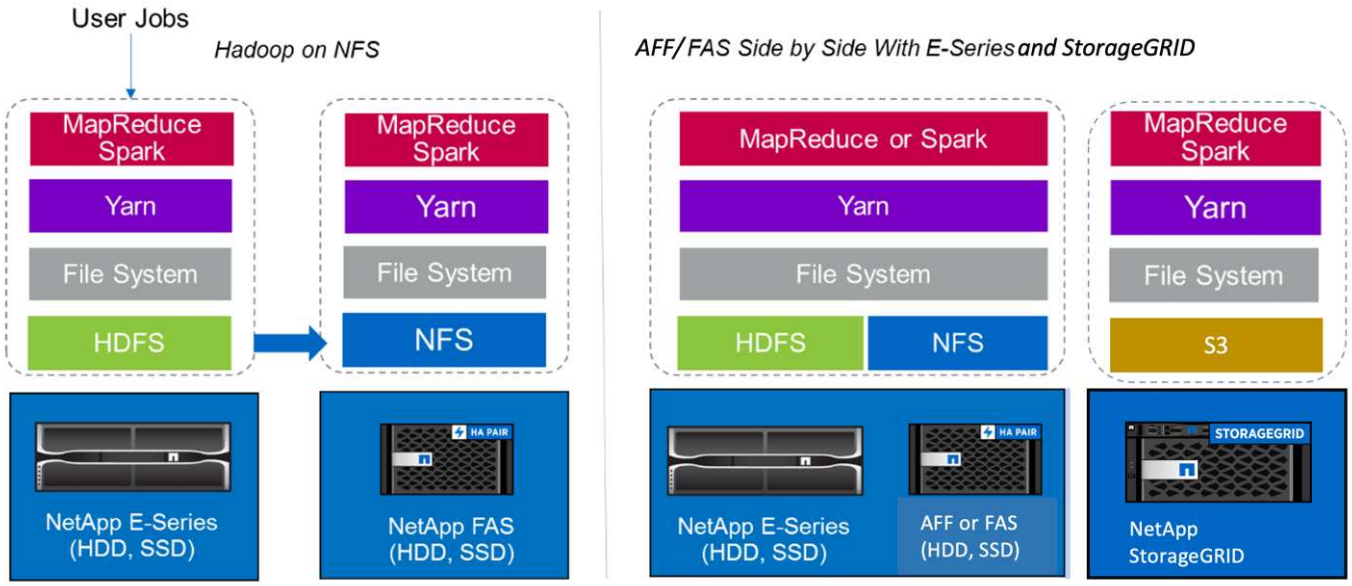


La solución ONTAP Spark utiliza el protocolo de acceso directo NFS de NetApp para análisis in situ y flujos de trabajo de IA, ML y DL mediante el acceso a los datos de producción existentes. Los datos de producción disponibles en los nodos de Hadoop se exportan para realizar trabajos de análisis e IA sin movimiento, ML y DL. Puede acceder a los datos para procesar en nodos Hadoop con el acceso directo NFS de NetApp o sin ellos. En Spark con el independiente o. `yarn` Gestor de clústeres, puede configurar un volumen NFS mediante `file:///<target_volume`. Validamos tres casos de uso con conjuntos de datos diferentes. Los detalles de estas validaciones se presentan en la sección “resultados de la prueba”. (xref)

La siguiente figura muestra el posicionamiento del almacenamiento de Apache Spark/Hadoop de NetApp.



Hemos identificado las características únicas de la solución Spark E-Series, la solución AFF/FAS ONTAP Spark y la solución StorageGRID Spark, y se ha realizado una validación y pruebas detalladas. Basándose en nuestras observaciones, NetApp recomienda la solución E-Series para instalaciones vírgenes y puestas en marcha escalables nuevas y la solución AFF/FAS para cargas de trabajo de análisis in situ, IA, ML y DL que utilizan datos NFS existentes, y StorageGRID para IA, APRENDIZAJE AUTOMÁTICO y análisis de datos modernos cuando se requiere almacenamiento de objetos.



Un lago de datos es un repositorio de almacenamiento para grandes conjuntos de datos de forma nativa que se puede usar para trabajos de análisis, IA, ML y DL. Creamos un repositorio de lago de datos para las soluciones E-Series, AFF/FAS y SG6060 de StorageGRID. El sistema E-Series proporciona acceso HDFS al clúster de Hadoop Spark, mientras que se accede a los datos de producción existentes a través del protocolo de acceso directo NFS al clúster de Hadoop. Para los conjuntos de datos que residen en el almacenamiento de objetos, StorageGRID de NetApp proporciona acceso seguro S3 y S3A.

"Siguiente: Resumen de casos de uso."

Resumen de casos de uso

"Anterior: Información general de las soluciones Spark de NetApp."

Transmisión de datos

Apache Spark puede procesar datos en streaming, que se utilizan para extraer, transformar y cargar en streaming procesos (ETL); enriquecimiento de datos; activación de la detección de eventos; y análisis de sesiones complejas:

- **Streaming ETL.** los datos se limpian y se agregan continuamente antes de que se insertan en almacenes de datos. Netflix usa las secuencias de Kafka y Spark para crear una solución en línea de recomendación de películas y supervisión de datos en tiempo real que puede procesar miles de millones de eventos al día de diferentes fuentes de datos. Sin embargo, las ETL tradicionales para el procesamiento por lotes se tratan de forma distinta. Estos datos se leen primero, y luego se convierten en un formato de base de datos antes de ser escritos en la base de datos.
- **Enriquecimiento de datos.** la transmisión de Spark enriquece los datos en directo con datos estáticos para permitir un análisis de datos en tiempo real más completo. Por ejemplo, los anunciantes en línea pueden entregar anuncios personalizados y orientados dirigidos por información sobre el comportamiento del cliente.
- **Detección de sucesos de disparo.** la transmisión de Spark le permite detectar y responder rápidamente a comportamientos inusuales que podrían indicar problemas potencialmente graves. Por ejemplo, las instituciones financieras utilizan desencadenantes para detectar y detener las transacciones de fraude, y los hospitales utilizan desencadenadores para detectar cambios sanitarios peligrosos detectados en los signos vitales de un paciente.

- **Análisis complejo de sesiones.** Spark Streaming recopila eventos como la actividad del usuario después de iniciar sesión en un sitio web o aplicación, que se agrupan y analizan a continuación. Por ejemplo, Netflix utiliza esta funcionalidad para proporcionar recomendaciones de películas en tiempo real.

Para obtener más información sobre la configuración de los datos en streaming, la verificación de Confluent Kafka y las pruebas de rendimiento, consulte "TR-4912: Directrices de mejores prácticas para el almacenamiento por niveles de Confluent Kafka con NetApp".

Aprendizaje automático

El marco integrado de Spark le ayuda a ejecutar consultas repetidas en conjuntos de datos mediante la biblioteca de aprendizaje automático (MLlib). MLlib se utiliza en áreas tales como agrupación en clúster, clasificación y reducción de la dimensionalidad para algunas funciones comunes de big data como inteligencia predictiva, segmentación del cliente para fines de marketing y análisis de sentimiento. MLlib se utiliza en la seguridad de la red para realizar inspecciones en tiempo real de los paquetes de datos en busca de indicaciones de actividad maliciosa. Ayuda a los proveedores de seguridad a conocer las nuevas amenazas y a mantenerse por delante de los hackers mientras protegen a sus clientes en tiempo real.

Aprendizaje profundo

TensorFlow es un marco de aprendizaje profundo más popular que se utiliza en el sector. TensorFlow admite el entrenamiento distribuido en un clúster de CPU o GPU. Este entrenamiento distribuido permite a los usuarios ejecutarlo en una gran cantidad de datos con muchas capas profundas.

Hasta hace poco, si queríamos utilizar TensorFlow con Apache Spark, teníamos que realizar todas las operaciones ETL necesarias para TensorFlow en PySpark y luego escribir datos en el almacenamiento intermedio. Esos datos se cargarán en el clúster TensorFlow para el proceso de entrenamiento real. Este flujo de trabajo requería que el usuario mantuviera dos clústeres diferentes: Uno para ETL y otro para el entrenamiento distribuido de TensorFlow. Ejecutar y mantener varios clústeres era normalmente tedioso y laborioso.

DataFrames y RDD en versiones anteriores de Spark no estaban bien adaptados para el aprendizaje profundo debido a que el acceso aleatorio era limitado. En Spark 3.0 con hidrógeno del proyecto, se añade soporte nativo a los marcos de aprendizaje profundo. Este enfoque permite la programación basada en MapReduce en el clúster de Spark.

Análisis interactivo

Apache Spark es lo suficientemente rápido como para realizar consultas exploratorias sin necesidad de muestreo con idiomas de desarrollo distintos de Spark, incluidas SQL, R y Python. Spark utiliza herramientas de visualización para procesar datos complejos y visualizarlos de forma interactiva. Spark with Structured streaming realiza consultas interactivas contra datos en directo en análisis web que le permiten ejecutar consultas interactivas en la sesión actual de un visitante web.

Sistema de recomendación

Con el paso de los años, los sistemas de recomendación han traído consigo enormes cambios a nuestras vidas, ya que las empresas y los consumidores han respondido a los cambios drásticos en las compras en línea, el entretenimiento en línea y muchos otros sectores. De hecho, estos sistemas se encuentran entre los casos de éxito más evidentes de la IA en la producción. En muchos casos de uso práctico, los sistemas de recomendación se combinan con IA conversacional o bots conversacionales interpuestos con un back-end NLP para obtener información relevante y producir inferencias útiles.

Hoy en día, muchos minoristas están adoptando nuevos modelos de negocios como comprar en línea y

recoger en la tienda, recoger en la acera, hacer el autocheck-out, escanear y llevar, entre otros. Estos modelos se han convertido en algo más destacado durante la pandemia del COVID-19, al hacer que las compras sean más seguras y convenientes para los consumidores. La IA es crucial para estas tendencias digitales en aumento, que están influenciadas por el comportamiento de los consumidores y viceversa. Para satisfacer las crecientes demandas de los consumidores, con el fin de aumentar la experiencia del cliente, mejorar la eficiencia operativa y aumentar los ingresos, NetApp ayuda a sus clientes empresariales y empresas a usar algoritmos de aprendizaje automático y aprendizaje profundo para diseñar sistemas de recomendación más rápidos y precisos.

Existen varias técnicas utilizadas para ofrecer recomendaciones, como el filtrado colaborativo, los sistemas basados en contenido, el modelo de recomendación de aprendizaje profundo (DLRM) y las técnicas híbridas. Anteriormente, los clientes utilizaban PySpark para implementar un filtrado colaborativo para crear sistemas de recomendación. Spark MLlib implementa cuadrados alternos mínimos (ALS) para el filtrado colaborativo, un algoritmo muy popular entre las empresas antes del ascenso de DLRM.

Procesamiento de lenguaje natural

La IA conversacional, posible gracias al procesamiento del lenguaje natural (NLP), es la rama de la IA que ayuda a los ordenadores a comunicarse con los seres humanos. NLP tiene una gran prevalencia en todos los sectores verticales y en muchos casos de uso, desde asistentes inteligentes y bots conversacionales hasta búsquedas en Google y texto predictivo. Según a "Informe de Gartner" Predicción, para 2022, el 70 % de las personas interactuará con plataformas de IA conversacionales diariamente. Para mantener una conversación de alta calidad entre un ser humano y una máquina, las respuestas deben ser rápidas, inteligentes y naturales.

Los clientes necesitan una gran cantidad de datos para procesar y entrenar sus modelos NLP y de reconocimiento automático de voz (ASR). También necesitan mover datos en el perímetro, el núcleo y el cloud, y necesitan poder realizar inferencia en milisegundos para establecer una comunicación natural con las personas. NetApp AI y Apache Spark es una combinación ideal de computación, almacenamiento, procesamiento de datos, entrenamiento de modelos, ajuste preciso e implementación.

El análisis de sentimiento es un campo de estudio dentro de las BPL en el que se extraen sentimientos positivos, negativos o neutrales del texto. El análisis de confianza tiene una variedad de casos de uso, desde determinar el rendimiento del empleado del centro de soporte en conversaciones con personas que llaman hasta proporcionar respuestas automatizadas apropiadas al bot conversacional. También se ha utilizado para predecir el precio de las acciones de una empresa en función de las interacciones entre representantes de la empresa y la audiencia en llamadas trimestrales sobre ganancias. Además, el análisis de confianza se puede utilizar para determinar la visión de un cliente sobre los productos, servicios o asistencia proporcionados por la Marca.

Utilizamos la "Chispa NLP" biblioteca desde "John Snow Labs" Cargar tuberías preentrenadas y Representaciones bidireccionales de Encoder desde los modelos Transformers (BERT) incluyendo "confianza en las noticias financieras" y.. "FinBERT", realizando tokenización, reconocimiento de entidades con nombre, entrenamiento modelo, ajuste y análisis de sentimiento a escala. Spark NLP es la única biblioteca de código abierto de NLP en producción que ofrece transformadores de última generación como BERT, ALBERT, ELECTRA, XLNet, DistilBERT, Roberta, DeBERTa, XLM- Roberta, Longexer, ELMO, Codificador universal de frases, Google T5, MarianMT y GPT2. La biblioteca funciona no sólo en Python y R, sino también en el ecosistema JVM (Java, Scala y Kotlin) a escala ampliando Apache Spark de forma nativa.

"Siguiente: Principales casos prácticos de IA, APRENDIZAJE AUTOMÁTICO y aprendizaje profundo."

Principales casos de uso y arquitecturas de IA, APRENDIZAJE AUTOMÁTICO y aprendizaje profundo

"Anterior: Resumen de casos de uso."

Los principales casos de uso y metodología de IA, APRENDIZAJE AUTOMÁTICO y aprendizaje profundo pueden dividirse en las siguientes secciones:

Canalizaciones NLP Spark y la inferencia distribuida TensorFlow

La siguiente lista contiene las bibliotecas NLP de código abierto más populares que han adoptado la comunidad de ciencias de datos bajo diferentes niveles de desarrollo:

- "Kit de herramientas de lenguaje natural (NLMK)". El kit de herramientas completo para todas las técnicas de NLP. Se mantiene desde principios de los años 2000.
- "TextBlob". Una herramienta NLP de fácil uso API de Python construida sobre NLMK y Pattern.
- "Stanford Core NLP". Servicios y paquetes NLP en Java desarrollados por el Grupo NLP de Stanford.
- "Gensim". Modelado de Temas para humanos comenzó como una colección de scripts de Python para el proyecto de la Biblioteca Checa de Matemáticas digitales.
- "La piratería". Flujos de trabajo NLP industrial integrales con Python y Cython con aceleración de GPU para transformadores.
- "Fasttext". Una biblioteca gratuita, ligera y de código abierto de NLP para el aprendizaje de textos y clasificación de frases creada por el laboratorio DE investigación DE IA (FAIR) de Facebook.

Spark NLP es una solución única y unificada para todas las tareas y requisitos de NLP que permite un software escalable, de alto rendimiento y de alta precisión impulsado por NLP para casos de uso reales de producción. Aprovecha el aprendizaje de transferencia e implementa los últimos algoritmos y modelos de última generación en investigación y en diferentes industrias. Debido a la falta de pleno apoyo por parte de Spark para las bibliotecas anteriores, Spark NLP se creó sobre la base de "Estimular ML" Aprovechar el motor de procesamiento de datos distribuidos en memoria de uso general de Spark como biblioteca de NLP de clase empresarial para flujos de trabajo de producción esenciales. Sus anotadores utilizan algoritmos basados en reglas, aprendizaje automático y TensorFlow para impulsar las implementaciones de aprendizaje profundo. Esto cubre las tareas comunes de las NLP, incluyendo, entre otras cosas, la tokenización, la limmatización, la secución, el marcado parcial del habla, el reconocimiento de entidades denominadas, revisión ortográfica y análisis de confianza.

Representaciones del codificador bidireccional de Transformers (BERT) es una técnica de aprendizaje automático basada en transformadores para NLP. Popularizó el concepto de preentrenamiento y ajuste fino. La arquitectura del transformador en BERT se originó a partir de la traducción automática, que modela las dependencias a largo plazo mejor que los modelos de lenguaje basados en Red neuronal recurrente (RNN). También introdujo la tarea Modelización de Lenguaje Mask (MLM), donde un 15% aleatorio de todos los tokens están enmascarados y el modelo los predice, permitiendo una verdadera bidirectionality.

El análisis de la confianza financiera es un reto debido al lenguaje especializado y a la falta de datos etiquetados en ese dominio. "FinBERT", Un modelo de idioma basado en BERT preformado, fue adaptado el dominio en "Reuters TRC2", un corpus financiero y ajustado con datos etiquetados ("Banco de frases financieras") para la clasificación de la confianza financiera. Los investigadores extrajeron 4, 500 frases de artículos de noticias con términos financieros. Luego 16 expertos y maestros estudiantes con fondos financieros etiquetaron las frases como positivas, neutrales y negativas. Hemos creado un flujo de trabajo completo de Spark para analizar el sentimiento de las transcripciones de llamadas de beneficios de empresas del mejor de los 10 NASDAQ de 2016 a 2020 usando FinBERT y otros dos oleoductos preentrenados (

"Análisis de confianza para las noticias financieras", "Explicar el documento DL") Desde Spark NLP.

El motor de aprendizaje profundo subyacente para Spark NLP es TensorFlow, una plataforma integral de código abierto para el aprendizaje automático que permite una creación de modelos fácil, una producción de ML sólida en cualquier lugar y una experimentación potente para la investigación. Por lo tanto, al ejecutar nuestras tuberías en Spark `yarn cluster` Modo, estábamos ejecutando TensorFlow distribuido con datos y paralelización de modelos en un nodo maestro y varios de trabajadores, además de almacenamiento conectado a la red montado en el clúster.

Horovod distribuyó entrenamiento

La validación principal de Hadoop para el rendimiento relacionado con MapReduce se realiza con TeraGen, TeraSort, TeraValidate y DFSIO (lectura y escritura). Los resultados de la validación TeraGen y TeraSort se presentan en "TR-3969: Soluciones de NetApp para Hadoop" Para E-Series y en la sección “Storage Tiering” (xref) para AFF.

En función de las solicitudes de los clientes, consideramos que la formación distribuida con Spark es una de las más importantes de las distintas situaciones de uso. En este documento, utilizamos la "Hovorod en Spark" Para validar el rendimiento de Spark con NetApp en las instalaciones, nativas del cloud e híbridas mediante las controladoras de almacenamiento All Flash FAS (AFF) de NetApp, Azure NetApp Files y StorageGRID.

El paquete Horovod en Spark proporciona un contenedor adecuado en torno a Horovod que simplifica la ejecución de cargas de trabajo de entrenamiento distribuidas en clústeres de Spark, lo cual permite crear un bucle de diseño de modelo ajustado en el que el procesamiento de datos, el entrenamiento de modelos y la evaluación de modelos se llevan a cabo en Spark donde se encuentran los datos de entrenamiento e inferencia.

Hay dos API para ejecutar Horovod en Spark: Una API de Estimator de alto nivel y una API de Run de bajo nivel. Aunque ambos usan el mismo mecanismo subyacente para lanzar Horovod en los ejecutores de Spark, la API de Estimator abstrae el procesamiento de datos, el bucle de entrenamiento del modelo, la verificación del modelo, la recopilación de métricas y la formación distribuida. Utilizamos Horovod Spark Estimators, TensorFlow y Keras para una preparación de datos completa y un flujo de trabajo de entrenamiento distribuido basado en la "Ventas de la tienda Kaggle Rossmann" competencia.

El script `keras_spark_horovod_rossmann_estimator.py` puede encontrarse en la sección "Scripts Python para cada caso de uso principal." Contiene tres partes:

- La primera parte realiza varios pasos de preprocesamiento de datos sobre un conjunto inicial de archivos CSV proporcionados por Kaggle y reunidos por la comunidad. Los datos de entrada se separan en un grupo de entrenamiento con un `Validation` un subconjunto y un conjunto de datos de pruebas.
- La segunda parte define un modelo de red neuronal profunda Keras (DNN) con función de activación sigmoide logarítmica y un optimizador Adam, y realiza un entrenamiento distribuido del modelo mediante Horovod en Spark.
- La tercera parte realiza predicciones en el conjunto de datos de pruebas utilizando el mejor modelo que minimiza el error absoluto de la media general del conjunto de validación. A continuación, crea un archivo CSV de salida.

Consulte la sección ""“Aprendizaje automático”" para varios resultados de comparación de tiempo de ejecución.

Aprendizaje profundo para múltiples trabajadores usando Keras para la predicción CTR

Con los recientes avances en las plataformas Y aplicaciones DE ML, ahora se presta mucha atención al

aprendizaje a escala. La tasa de clics (CTR) se define como el número medio de clics-throughs por cien impresiones de anuncios en línea (expresado como porcentaje). Se ha adoptado ampliamente como métrica clave en diversos mercados verticales del sector y casos de uso, incluidos el marketing digital, el comercio minorista, el comercio electrónico y los proveedores de servicios. Consulte nuestra "TR-4904: Formación distribuida en Azure - predicción de frecuencias mediante clic" Para obtener más información sobre las aplicaciones de CTR y una implementación de flujo de trabajo integral de Cloud AI con Kubernetes, ETL de datos distribuidos y entrenamiento de modelos con DASK y CUDA ML.

En este informe técnico se utilizó una variación del "Criteo Terabyte haga clic en el conjunto de datos Logs" (Consulte TR-4904) para el aprendizaje profundo distribuido por varios trabajadores mediante Keras para crear un flujo de trabajo de Spark con modelos Deep and Cross Network (DCN), donde se compara su rendimiento en términos de una función de error de pérdida de registro con un modelo de regresión logística de Spark ML básico. DCN capta eficazmente las interacciones eficaces de funciones de grados limitados, aprende interacciones muy no lineales, no requiere ingeniería de funciones manuales ni búsqueda exhaustiva y tiene un bajo coste computacional.

Los datos para los sistemas de recomendación a escala web son en su mayoría discretos y categóricos, lo que conduce a un gran espacio de funciones escasas que resulta difícil para la exploración de funciones. Esto ha limitado la mayoría de los sistemas a gran escala a modelos lineales como la regresión logística. Sin embargo, la clave para hacer buenas predicciones es identificar las características predictivas más frecuentes y, al mismo tiempo, explorar características cruzadas poco visibles o raras. Los modelos lineales son simples, interpretables y fáciles de escalar, pero están limitados en su poder expresivo.

Por otro lado, se ha demostrado que las características cruzadas son significativas para mejorar la expresividad de los modelos. Desafortunadamente, a menudo requiere ingeniería de funciones manual o búsqueda exhaustiva para identificar estas funciones. Generalizar a interacciones de características no vistas es a menudo difícil. El uso de una red neuronal cruzada como DCN evita la ingeniería de funciones específicas de una tarea mediante la aplicación explícita de un cruce de funciones de forma automática. La red cruzada consta de varias capas, donde el grado más alto de interacciones se determina de forma comprobable por la profundidad de la capa. Cada capa produce interacciones de mayor orden basadas en las existentes y mantiene las interacciones de capas anteriores.

Una red neuronal profunda (DNN) tiene la promesa de capturar interacciones muy complejas entre características. Sin embargo, en comparación con DCN, requiere casi un orden de magnitud más parámetros, no es capaz de formar características cruzadas explícitamente y puede no aprender eficazmente algunos tipos de interacciones de características. La red cruzada es eficiente en la memoria y fácil de implementar. El entrenamiento conjunto de los componentes cross y DNN captura de forma eficiente las interacciones predictivas con las características y proporciona un rendimiento de última generación en el conjunto de datos Criteo CTR.

Un modelo DCN comienza con una capa de incrustación y apilado, seguido por una red transversal y una red profunda en paralelo. A su vez, están seguidas por una última capa de combinación que combina las salidas de las dos redes. Los datos de entrada pueden ser un vector con características dispersas y densas. En Spark, ambas "ml" y.. "mllib" las bibliotecas contienen el tipo `SparseVector`. Por lo tanto, es importante que los usuarios distingan entre los dos y sean conscientes cuando llaman a sus respectivas funciones y métodos. En sistemas de recomendación a escala web, como la predicción CTR, los insumos son, en su mayoría, características categóricas, por ejemplo `'country=usa'`. Tales características se codifican a menudo como vectores de una caliente, por ejemplo, `'[0,1,0, …]'`. Codificación en caliente (OHE) con `SparseVector` es útil cuando se trata de conjuntos de datos del mundo real con vocabularios en constante cambio y crecimiento. Hemos modificado ejemplos en "DeepCTR" Procesar grandes vocabularios, creando vectores de incrustación en la capa de incrustación y apilado de nuestro DCN.

La "Conjunto de datos de anuncios de visualización Criteo" predice la velocidad de clic del ads. Tiene 13 características de enteros y 26 características categóricas en las cuales cada categoría tiene una cardinalidad alta. Para este conjunto de datos, una mejora de 0.001 en pérdida de registro es prácticamente significativa

debido al gran tamaño de entrada. Una pequeña mejora en la precisión de la predicción para una gran base de usuarios puede conducir potencialmente a un gran aumento en los ingresos de una empresa. El conjunto de datos contiene 11 GB de registros de usuarios de un periodo de 7 días, lo que equivale a unos 41 millones de registros. Utilizamos Spark `dataFrame.randomSplit()function` dividir de forma aleatoria los datos para el entrenamiento (80%), la validación cruzada (10%) y el 10% restante para las pruebas.

DCN se implementó en TensorFlow con Keras. La aplicación del proceso de formación modelo con DCN consta de cuatro componentes principales:

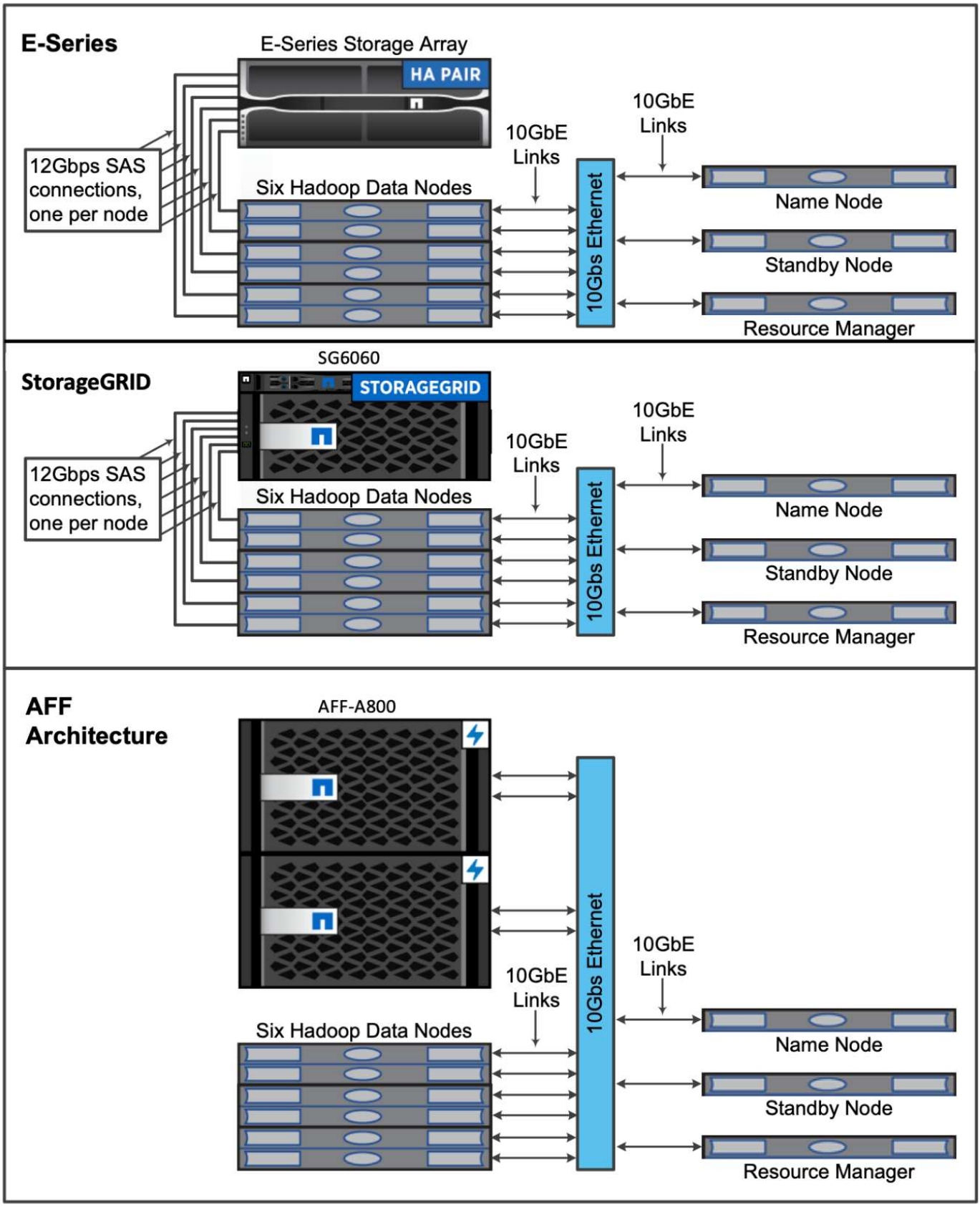
- **Procesamiento e incrustación de datos.** las características con valor real se normalizan aplicando una transformación de registro. Para las características categóricas, incrustamos las características en vectores densos de la dimensión 6×(cardinalidad de categoría)1/4. La concatenación de todos los embeddings da como resultado un vector de dimensión 1026.
- * Optimización.* aplicamos la optimización estocástica de minilotes con el optimizador Adam. El tamaño de lote se estableció en 512. La normalización por lotes se aplicó a la red profunda y la norma de clip degradado se estableció en 100.
- **Regularización.** utilizamos la parada temprana, ya que la regularización o el abandono L2 no se encontró para ser eficaz.
- * Hiperparámetros.* reportamos resultados basados en una búsqueda de cuadrícula sobre el número de capas ocultas, el tamaño de la capa oculta, la tasa de aprendizaje inicial y el número de capas cruzadas. El número de capas ocultas oscilaba entre 2 y 5, con un tamaño de capa oculto que oscilaba entre 32 y 1024. Para DCN, el número de capas cruzadas fue de 1 a 6. La tasa de aprendizaje inicial se ajustó de 0.0001 a 0.001 con incrementos de 0.0001. Todos los experimentos se pararon temprano en la etapa de entrenamiento 150,000, más allá de la cual empezó a ocurrir el ajuste excesivo.

Además de DCN, también hemos probado otros modelos populares de aprendizaje profundo para la predicción CTR, como "DeepFM", "XDeepFM", "AutoInt", y. "DCN v2".

Arquitecturas que se utilizan para la validación

Para esta validación, hemos utilizado cuatro nodos de trabajo y uno de nodo maestro con un par de alta disponibilidad de AFF-A800. Todos los miembros del clúster se conectaron mediante switches de red de 10 GbE.

Para esta validación de la solución Spark de NetApp, se utilizaron tres controladoras de almacenamiento distintas: El E5760, el E5724 y AFF-A800. Las controladoras de almacenamiento E-Series se conectaron a cinco nodos de datos con conexiones SAS de 12 Gbps. La controladora de almacenamiento de la pareja de alta disponibilidad de AFF proporciona volúmenes NFS exportados a través de conexiones 10 GbE a los nodos de trabajo de Hadoop. Los miembros de los clústeres de Hadoop se conectaron mediante conexiones 10 GbE en las soluciones E-Series, AFF y Hadoop de StorageGRID.



"Siguiente: Resultados de las pruebas."

Resultados de las pruebas

"Anterior: Principales casos prácticos de IA, ML y DL, y arquitecturas."

Se utilizaron los scripts TeraSort y TeraValidate en la herramienta de prueba de rendimiento TeraGen para medir la validación del rendimiento de Spark con las configuraciones E5760, E5724 y AFF-A800. Además se probaron tres casos de uso principales: Las canalizaciones SPARK NLP y el entrenamiento distribuido TensorFlow, el entrenamiento distribuido Horovod y el aprendizaje profundo multi-trabajador usando Keras para la predicción de CTR con DeepFM.

Para la validación E-Series y StorageGRID, utilizamos el factor de replicación de Hadoop 2. Para la validación AFF, solo utilizamos una fuente de datos.

En la siguiente tabla se enumera la configuración de hardware para la validación del rendimiento de Spark.

Tipo	Nodos de trabajo de Hadoop	Tipo de unidad	Unidades por nodo	Controladora de almacenamiento
SG6060	4	SAS	12	Un único par de alta disponibilidad
E5760	4	SAS	60	Un único par de alta disponibilidad
E5724	4	SAS	24	Un único par de alta disponibilidad
AFF800	4	SSD	6	Un único par de alta disponibilidad

En la siguiente tabla se enumeran los requisitos de software.

De NetApp	Versión
RHEL	7.9
Entorno de ejecución de OpenJDK	1.8.0
Servidor OpenJDK VM de 64 bits	25.302
Git	2.24.1
GCC/G++	11.2.1
Chispa	3.2.1
PySpark	3.1.2
SparkNLP	3.4.2
TensorFlow	2.9.0
Keras	2.9.0
Horovod	0.24.3

Análisis de la confianza financiera

Publicamos "TR-4910: Análisis de sentimiento en las comunicaciones de los clientes con IA de NetApp", En el

que se ha creado una canalización de IA conversacional integral con el "Kit de herramientas de operaciones de datos de NetApp", Almacenamiento AFF y NVIDIA DGX System. La canalización realiza el procesamiento de señales de audio por lotes, el reconocimiento automático de voz (ASR), el aprendizaje de transferencia y el análisis de sentimiento con el conjunto de herramientas DataOPS, "SDK de NVIDIA Riva", y la "Marco Tao". Ampliando el caso de uso del análisis de sentimiento al sector de los servicios financieros, construimos un flujo de trabajo SparkNLP, cargamos tres modelos BERT para diversas tareas de NLP, como el reconocimiento de entidades con nombre, y obtuvimos un sentimiento a nivel de frase para las llamadas trimestrales de beneficios de las empresas incluidas en el NASDAQ Top 10.

El siguiente script `sentiment_analysis_spark. py` Utiliza el modelo FinBERT para procesar transcripciones en HDFS y producir recuentos de sentimientos positivos, neutros y negativos, como se muestra en la siguiente tabla:

```
-bash-4.2$ time ~/anaconda3/bin/spark-submit
--packages com.johnsnowlabs.nlp:spark-nlp_2.12:3.4.3
--master yarn
--executor-memory 5g
--executor-cores 1
--num-executors 160
--conf spark.driver.extraJavaOptions="-Xss10m -XX:MaxPermSize=1024M"
--conf spark.executor.extraJavaOptions="-Xss10m -XX:MaxPermSize=512M"
/sparkusecase/tr-4570-nlp/sentiment_analysis_spark.py
hdfs:///data1/Transcripts/
> ./sentiment_analysis_hdfs.log 2>&1
real13m14.300s
user557m11.319s
sys4m47.676s
```

En la tabla siguiente se enumera el análisis del sentimiento de «llamada a los beneficios» a nivel de frase de las empresas NASDAQ Top 10 de 2016 a 2020.

Recuentos de confianza y porcentaje	**Todas las 10 empresas**	**AAPL**	**AIRBUS**	**AMZN**	**CSCO**	**GOOGL**	**INTC**	**MSFT**	**NVDA**
Recuentos positivos	7447	1567	743	290	682	826	824	904	417
Recuentos de punto muerto	64067	6856	7596	5086	6650	5914	6099	5715	6189
Recuentos negativos	1787	253	213	84	189	97	282	202	89

Recuentos de confianza y porcentaje	Todas las 10 empresas	AAPL	AIRBUS	AMZN	CSCO	GOOGL	INTC	MSFT	NVDA
Conteos sin categorizar	196	0	0	76	0	0	0	1	0
(recuentos totales)	73497	8676	8552	5536	7521	6837	7205	6822	6695

En términos de porcentajes, la mayoría de las sentencias pronunciadas por los CEOs y CFO son fácticas y, por lo tanto, tienen un sentimiento neutral. Durante una llamada de beneficios, los analistas hacen preguntas que podrían transmitir un sentimiento positivo o negativo. Vale la pena investigar cuantitativamente cómo la confianza negativa o positiva afecta los precios de las acciones el mismo día o día siguiente de las operaciones.

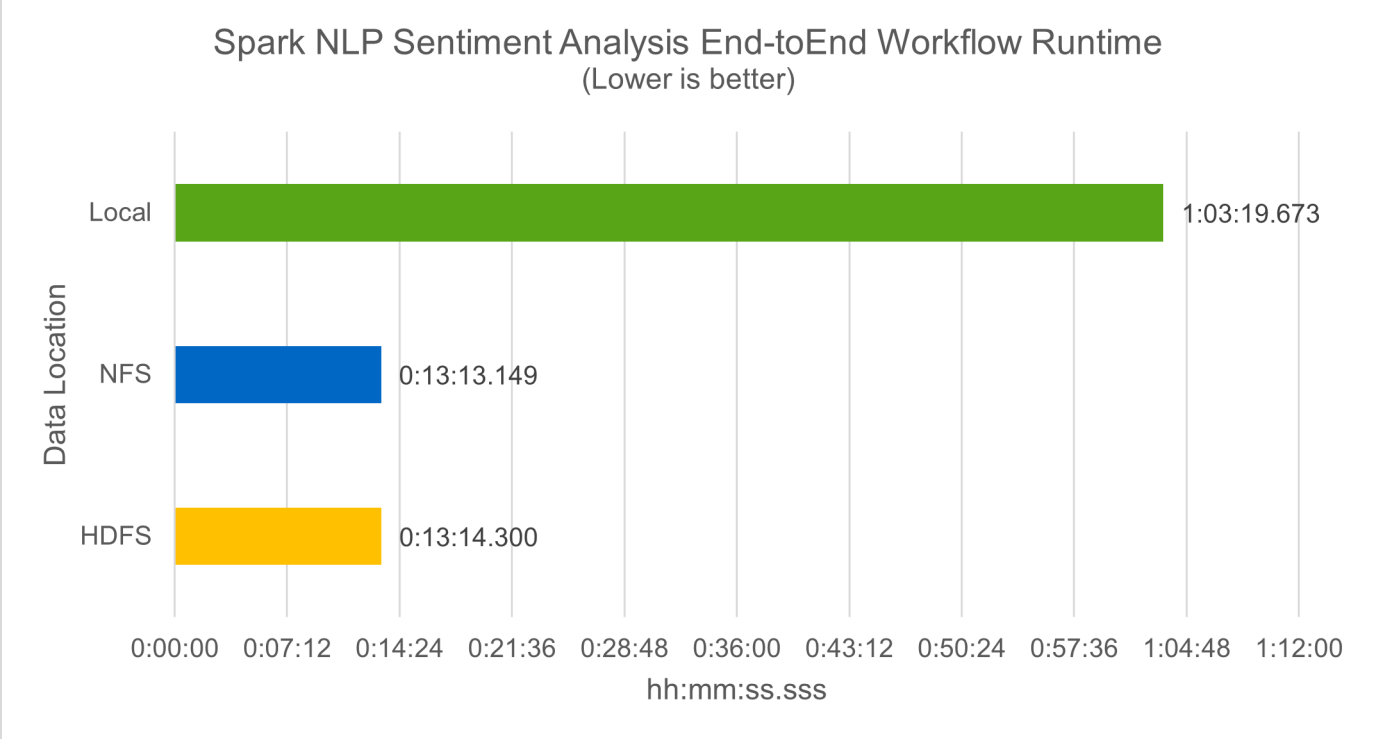
En la siguiente tabla se enumera el análisis de sentimiento a nivel de frase para las empresas NASDAQ Top 10, expresado en porcentaje.

Porcentaje de confianza	Todas las 10 empresas	AAPL	AIRBUS	AMZN	CSCO	GOOGL	INTC	MSFT	NVDA
Positivo	10.13 %	18.06 %	8.69 %	5.24 %	9.07 %	12.08 %	11.44 %	13.25 %	6.23 %
Neutro	87.17 %	79.02 %	88.82 %	91.87 %	88.42 %	86.50 %	84.65 %	83.77 %	92.44 %
Negativo	2.43 %	2.92 %	2.49 %	1.52 %	2.51 %	1.42 %	3.91 %	2.96 %	1.33 %
Sin clasificar	0.27 %	0 %	0 %	1.37 %	0 %	0 %	0 %	0.01 %	0 %

En cuanto al tiempo de ejecución del flujo de trabajo, observamos una mejora significativa de 4.78x con respecto a `local` Mode a un entorno distribuido en HDFS y una mejora adicional del 0.14 % gracias al aprovechamiento de NFS.

```
-bash-4.2$ time ~/anaconda3/bin/spark-submit
--packages com.johnsnowlabs.nlp:spark-nlp_2.12:3.4.3
--master yarn
--executor-memory 5g
--executor-cores 1
--num-executors 160
--conf spark.driver.extraJavaOptions="-Xss10m -XX:MaxPermSize=1024M"
--conf spark.executor.extraJavaOptions="-Xss10m -XX:MaxPermSize=512M"
/sparkusecase/tr-4570-nlp/sentiment_analysis_spark.py
file:///sparkdemo/sparknlp/Transcripts/
> ./sentiment_analysis_nfs.log 2>&1
real13m13.149s
user537m50.148s
sys4m46.173s
```

Como se muestra en la siguiente figura, el paralelismo de datos y modelos mejoró el procesamiento de datos y la velocidad de inferencia del modelo TensorFlow distribuido. La ubicación de los datos en NFS produjo un tiempo de ejecución ligeramente mejor porque el cuello de botella del flujo de trabajo es la descarga de modelos preentrenados. Si aumentamos el tamaño de los conjuntos de datos de transcripciones, la ventaja de NFS es más evidente.



Distribuye la formación con el rendimiento de Horovod

El siguiente comando produjo información de tiempo de ejecución y un archivo de registro en nuestro clúster de Spark mediante una sola `master` nodo con 160 ejecutores cada uno con un núcleo. La memoria del ejecutor estaba limitada a 5 GB para evitar errores de memoria insuficiente. Consulte la sección "“Guiones de Python para cada caso de uso principal”" para obtener más información sobre el procesamiento de datos, el

entrenamiento de modelos y el cálculo de precisión de modelos en `keras_spark_horovod_rossmann_estimator.py`.

```
(base) [root@n138 horovod]# time spark-submit
--master local
--executor-memory 5g
--executor-cores 1
--num-executors 160
/sparkusecase/horovod/keras_spark_horovod_rossmann_estimator.py
--epochs 10
--data-dir file:///sparkusecase/horovod
--local-submission-csv /tmp/submission_0.csv
--local-checkpoint-file /tmp/checkpoint/
> /tmp/keras_spark_horovod_rossmann_estimator_local. log 2>&1
```

El tiempo de ejecución resultante con diez épocas de entrenamiento fue el siguiente:

```
real43m34.608s
user12m22.057s
sys2m30.127s
```

Se necesitaron más de 43 minutos para procesar datos de entrada, entrenar un modelo DNN, calcular la precisión y generar puntos de control TensorFlow y un archivo CSV para resultados de predicción. Limitamos el número de épocas de entrenamiento a 10, que en la práctica a menudo se establece a 100 para garantizar una precisión satisfactoria del modelo. El tiempo de entrenamiento se escala linealmente con el número de épocas.

A continuación, utilizamos los cuatro nodos de trabajo disponibles en el clúster y ejecutamos el mismo script en `yarn` Modo con datos en HDFS:

```
(base) [root@n138 horovod]# time spark-submit
--master yarn
--executor-memory 5g
--executor-cores 1 --num-executors 160
/sparkusecase/horovod/keras_spark_horovod_rossmann_estimator.py
--epochs 10
--data-dir hdfs:///user/hdfs/tr-4570/experiments/horovod
--local-submission-csv /tmp/submission_1.csv
--local-checkpoint-file /tmp/checkpoint/
> /tmp/keras_spark_horovod_rossmann_estimator_yarn.log 2>&1
```

El tiempo de ejecución resultante se mejoró de la siguiente manera:

```
real8m13.728s
user7m48.421s
sys1m26.063s
```

Con el modelo y paralelismo de datos de Horovod en Spark, vimos una aceleración del tiempo de ejecución de 5.29x de `yarn` vs `local` modo con diez épocas de entrenamiento. Se muestra en la siguiente figura con las leyendas `HDFS` y.. `Local`. El entrenamiento del modelo TensorFlow DNN subyacente puede acelerarse más con GPU, si está disponible. Tenemos pensado llevar a cabo estas pruebas y publicar los resultados en un futuro informe técnico.

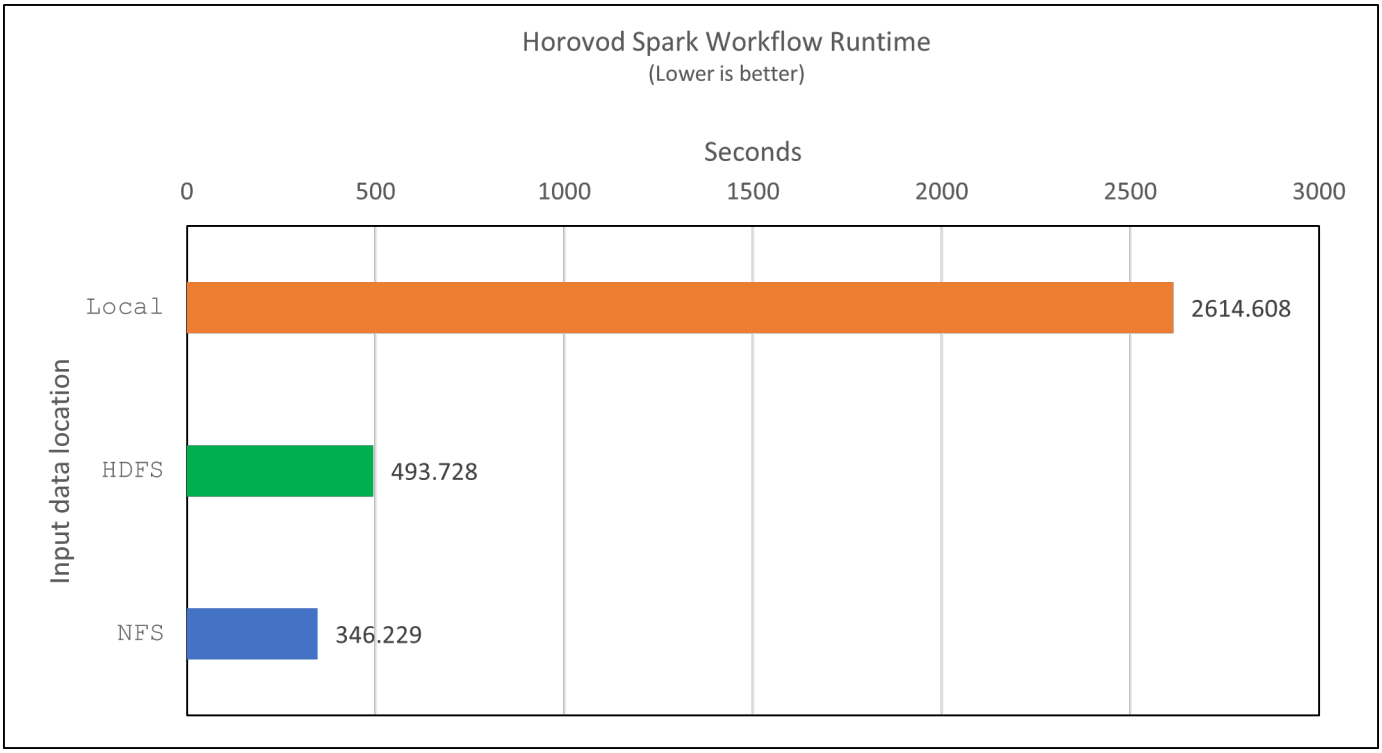
En nuestra siguiente prueba se compararon los tiempos de ejecución con los datos de entrada almacenados en NFS frente a HDFS. Se montó el volumen NFS en el AFF A800 `/sparkdemo/horovod` En los cinco nodos (un maestro, cuatro trabajadores) de nuestro clúster de Spark. Hicimos un comando similar al de pruebas anteriores, con el `--data- dir` Ahora el parámetro señala el montaje NFS:

```
(base) [root@n138 horovod]# time spark-submit
--master yarn
--executor-memory 5g
--executor-cores 1
--num-executors 160
/sparkusecase/horovod/keras_spark_horovod_rossmann_estimator.py
--epochs 10
--data-dir file:///sparkdemo/horovod
--local-submission-csv /tmp/submission_2.csv
--local-checkpoint-file /tmp/checkpoint/
> /tmp/keras_spark_horovod_rossmann_estimator_nfs.log 2>&1
```

El tiempo de ejecución resultante con NFS fue el siguiente:

```
real 5m46.229s
user 5m35.693s
sys  1m5.615s
```

Hubo otro 1,3x de aceleración, como se muestra en la siguiente figura. Por lo tanto, con un almacenamiento all-flash de NetApp conectado a su clúster, los clientes disfrutan de las ventajas de una transferencia y distribución de datos rápidas para flujos de trabajo de Horovod Spark, obteniendo una aceleración de 7,55 veces superior en comparación con ejecutarse en un único nodo.



Modelos de aprendizaje profundo para el rendimiento de la predicción CTR

Para sistemas de recomendación diseñados para maximizar CTR, debe aprender sofisticadas interacciones de funciones detrás de comportamientos de usuario que se pueden calcular matemáticamente desde orden bajo hasta orden alto. Tanto las interacciones de funciones de bajo orden como las de alto nivel deben ser igualmente importantes para un buen modelo de aprendizaje profundo sin orientarse hacia uno o hacia otro. DeepFM, una red neuronal basada en máquinas para la factorización profunda, combina máquinas de factorización para recomendaciones y aprendizaje profundo para el aprendizaje de funciones en una nueva arquitectura de redes neuronales.

Aunque las máquinas de factorización convencionales modelan las interacciones de funciones emparejadas como producto interno de vectores latentes entre las características y pueden teóricamente capturar información de alto orden, en la práctica, los profesionales de aprendizaje automático normalmente solo utilizan interacciones de funciones de segundo orden debido a la alta complejidad del almacenamiento y la computación. Variantes de redes neuronales profundas como las de Google "Modelos amplios profundos" por otro lado, aprende sofisticadas interacciones de características en una estructura de red híbrida combinando un modelo lineal amplio y un modelo profundo.

Hay dos entradas para este modelo ancho y profundo, uno para el modelo ancho subyacente y el otro para el profundo, la última parte de la cual todavía requiere ingeniería de características expertas y por lo tanto hace la técnica menos generalizable a otros dominios. A diferencia del modelo ancho y profundo, DeepFM puede ser entrenado eficientemente con características RAW sin ninguna ingeniería de características porque su gran parte y la parte profunda comparten la misma entrada y el vector de incrustación.

Primero procesamos el Criteo `train.txt` (11 GB) en un archivo CSV denominado `ctr_train.csv` Almacenados en un montaje NFS `/sparkdemo/tr-4570-data` uso `run_classification_criteo_spark.py` de la sección "“Guiones de Python para cada caso de uso principal”." Dentro de este script, la función `process_input_file` realiza varios métodos de cadena para quitar fichas e insertar `','` como delimitador y. `'\n'` como nueva línea. Tenga en cuenta que sólo necesita procesar el original `train.txt` una vez, para que el bloque de código se muestre como comentarios.

Para las siguientes pruebas de distintos modelos de aprendizaje profundo, utilizamos `ctr_train.csv` como archivo de entrada. En las pruebas posteriores, el archivo CSV de entrada se leyó en un Spark DataFrame con un esquema que contiene un campo de `'label'`, funciones densas de enteros `['I1', 'I2', 'I3', …, 'I13']`, y las características dispersas `['C1', 'C2', 'C3', …, 'C26']`. Lo siguiente `spark-submit` El comando toma en una entrada CSV, entrena modelos DeepFM con 20% de división para validación cruzada y elige el mejor modelo después de diez épocas de entrenamiento para calcular la precisión de predicción en el conjunto de pruebas:

```
(base) [root@n138 ~]# time spark-submit --master yarn --executor-memory 5g
--executor-cores 1 --num-executors 160
/sparkusecase/DeepCTR/examples/run_classification_criteo_spark.py --data
-dir file:///sparkdemo/tr-4570-data >
/tmp/run_classification_criteo_spark_local.log 2>&1
```

Tenga en cuenta que desde el archivo de datos `ctr_train.csv` Es superior a 11 GB, debe establecer un valor suficiente `spark.driver.maxResultSize` mayor que el tamaño del conjunto de datos para evitar errores.

```
 spark = SparkSession.builder \
    .master("yarn") \
    .appName("deep_ctr_classification") \
    .config("spark.jars.packages", "io.github.ravwojdyla:spark-schema-
utils_2.12:0.1.0") \
    .config("spark.executor.cores", "1") \
    .config('spark.executor.memory', '5gb') \
    .config('spark.executor.memoryOverhead', '1500') \
    .config('spark.driver.memoryOverhead', '1500') \
    .config("spark.sql.shuffle.partitions", "480") \
    .config("spark.sql.execution.arrow.enabled", "true") \
    .config("spark.driver.maxResultSize", "50gb") \
    .getOrCreate()
```

En el anterior `SparkSession.builder` la configuración también está activada "Flecha Apache", Que convierte un DataFrame de Spark en un DataFrame de Pandas con el `df.toPandas()` método.

```
22/06/17 15:56:21 INFO scheduler.DAGScheduler: Job 2 finished: toPandas at
/sparkusecase/DeepCTR/examples/run_classification_criteo_spark.py:96, took
627.126487 s
Obtained Spark DF and transformed to Pandas DF using Arrow.
```

Tras la división aleatoria, hay más de 36 M de filas en el conjunto de datos de entrenamiento y 9M de muestras en el conjunto de pruebas:

```
Training dataset size =  36672493
Testing dataset size =  9168124
```

Dado que este informe técnico se centra en las pruebas de CPU sin utilizar ninguna GPU, es imprescindible crear TensorFlow con indicadores adecuados del compilador. Este paso evita llamar a bibliotecas aceleradas por GPU y aprovecha al máximo las instrucciones de TensorFlow Advanced Vector Extensions (AVX) y AVX2. Estas características están diseñadas para cálculos algebraicos lineales como adición vectorizada, multiplicaciones de matrices dentro de un avance de alimentación o entrenamiento DNN de reproducción posterior. La instrucción Multiply Add (FMA) fusionada disponible con AVX2 utilizando registros de coma flotante de 256 bits (FP) es ideal para código entero y tipos de datos, lo que da como resultado una aceleración de hasta dos veces. En lo que respecta a los tipos de datos y código FP, AVX2 logra una aceleración del 8 % con respecto a AVX.

```
2022-06-18 07:19:20.101478: I
tensorflow/core/platform/cpu_feature_guard.cc:151] This TensorFlow binary
is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the
following CPU instructions in performance-critical operations:  AVX2 FMA
To enable them in other operations, rebuild TensorFlow with the
appropriate compiler flags.
```

Para crear TensorFlow a partir de origen, NetApp recomienda usar "Bazel". Para nuestro entorno, hemos ejecutado los siguientes comandos en el intérprete de comandos del shell para instalar `dnf`, `dnf-plugins`, Y Bazel.

```
yum install dnf
dnf install 'dnf-command(copr)'
dnf copr enable vbatts/bazel
dnf install bazel5
```

Debe habilitar GCC 5 o posterior para utilizar las funciones C++17 durante el proceso de compilación, que proporciona RHEL con la biblioteca de colecciones de software (SCL). Los siguientes comandos se instalan `devtoolset` Y GCC 11.2.1 en nuestro clúster RHEL 7.9:

```
subscription-manager repos --enable rhel-server-rhscl-7-rpms
yum install devtoolset-11-toolchain
yum install devtoolset-11-gcc-c++
yum update
scl enable devtoolset-11 bash
. /opt/rh/devtoolset-11/enable
```

Tenga en cuenta que los dos últimos comandos se habilitan `devtoolset-11`, que utiliza `/opt/rh/devtoolset-11/root/usr/bin/gcc` (GCC 11.2.1). También, asegúrese de que su `git` La versión es superior a 1.8.3 (se incluye con RHEL 7.9). Consulte este apartado "artículo" para la actualización `git` a 2.24.1.

Asumimos que ya ha clonado el repo maestro TensorFlow más reciente. A continuación, cree un `workspace` directorio con un `WORKSPACE` Archivo para crear TensorFlow a partir de origen con AVX, AVX2 y FMA. Ejecute el `configure` Y especifique la ubicación binaria Python correcta. "CUDA" Está deshabilitado para nuestras pruebas porque no utilizamos una GPU. A. `.bazelrc` el archivo se genera de acuerdo con su configuración. Además, hemos editado el archivo y configurado `build --define=no_hdfs_support=false` Para habilitar el soporte de HDFS. Consulte `.bazelrc` en la sección "“Guiones de Python para cada caso de uso principal”," para ver una lista completa de valores y marcas.

```
./configure
bazel build -c opt --copt=-mavx --copt=-mavx2 --copt=-mfma --copt=
-mfpmath=both -k //tensorflow/tools/pip_package:build_pip_package
```

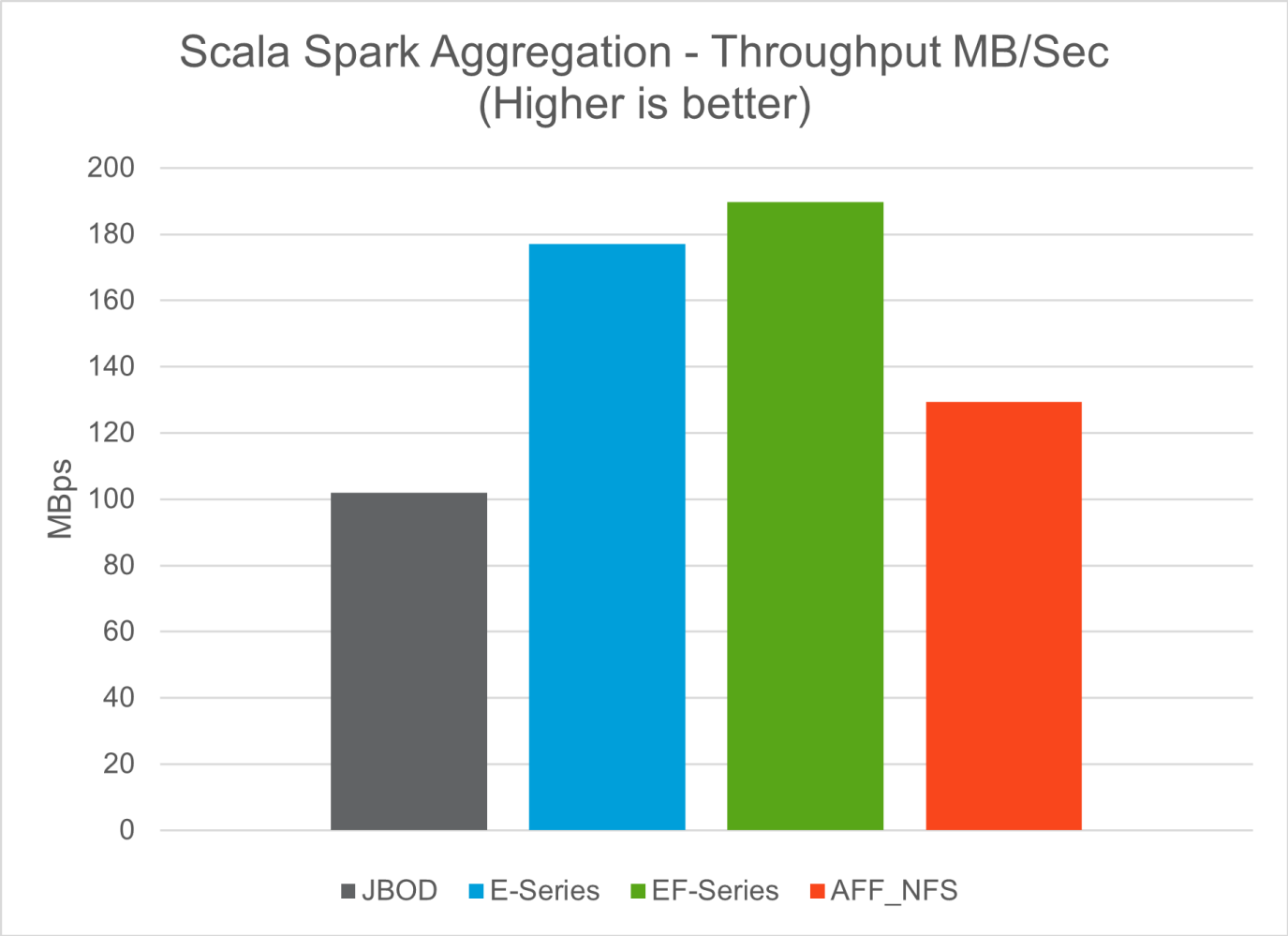
Después de crear TensorFlow con los indicadores correctos, ejecute la siguiente secuencia de comandos para procesar el conjunto de datos de anuncios de visualización Criteo, formar un modelo DeepFM y calcular el área bajo la curva de características operativas del receptor (AUC ROC) a partir de las puntuaciones de predicción.

```
(base) [root@n138 examples]# ~/anaconda3/bin/spark-submit
--master yarn
--executor-memory 15g
--executor-cores 1
--num-executors 160
/sparkusecase/DeepCTR/examples/run_classification_criteo_spark.py
--data-dir file:///sparkdemo/tr-4570-data
> . /run_classification_criteo_spark_nfs.log 2>&1
```

Después de diez épocas de entrenamiento, hemos obtenido la puntuación del AUC en el conjunto de datos de pruebas:

```
Epoch 1/10
125/125 - 7s - loss: 0.4976 - binary_crossentropy: 0.4974 - val_loss:
0.4629 - val_binary_crossentropy: 0.4624
Epoch 2/10
125/125 - 1s - loss: 0.3281 - binary_crossentropy: 0.3271 - val_loss:
0.5146 - val_binary_crossentropy: 0.5130
Epoch 3/10
125/125 - 1s - loss: 0.1948 - binary_crossentropy: 0.1928 - val_loss:
0.6166 - val_binary_crossentropy: 0.6144
Epoch 4/10
125/125 - 1s - loss: 0.1408 - binary_crossentropy: 0.1383 - val_loss:
0.7261 - val_binary_crossentropy: 0.7235
Epoch 5/10
125/125 - 1s - loss: 0.1129 - binary_crossentropy: 0.1102 - val_loss:
0.7961 - val_binary_crossentropy: 0.7934
Epoch 6/10
125/125 - 1s - loss: 0.0949 - binary_crossentropy: 0.0921 - val_loss:
0.9502 - val_binary_crossentropy: 0.9474
Epoch 7/10
125/125 - 1s - loss: 0.0778 - binary_crossentropy: 0.0750 - val_loss:
1.1329 - val_binary_crossentropy: 1.1301
Epoch 8/10
125/125 - 1s - loss: 0.0651 - binary_crossentropy: 0.0622 - val_loss:
1.3794 - val_binary_crossentropy: 1.3766
Epoch 9/10
125/125 - 1s - loss: 0.0555 - binary_crossentropy: 0.0527 - val_loss:
1.6115 - val_binary_crossentropy: 1.6087
Epoch 10/10
125/125 - 1s - loss: 0.0470 - binary_crossentropy: 0.0442 - val_loss:
1.6768 - val_binary_crossentropy: 1.6740
test AUC 0.6337
```

De un modo similar a los casos de uso anteriores, comparamos el tiempo de ejecución del flujo de trabajo de Spark con los datos alojados en diferentes ubicaciones. En la siguiente figura, se muestra una comparación de la predicción del CTR de aprendizaje profundo para un tiempo de ejecución de los flujos de trabajo de Spark.



"Siguiente: Solución de cloud híbrido."

Solución de cloud híbrido

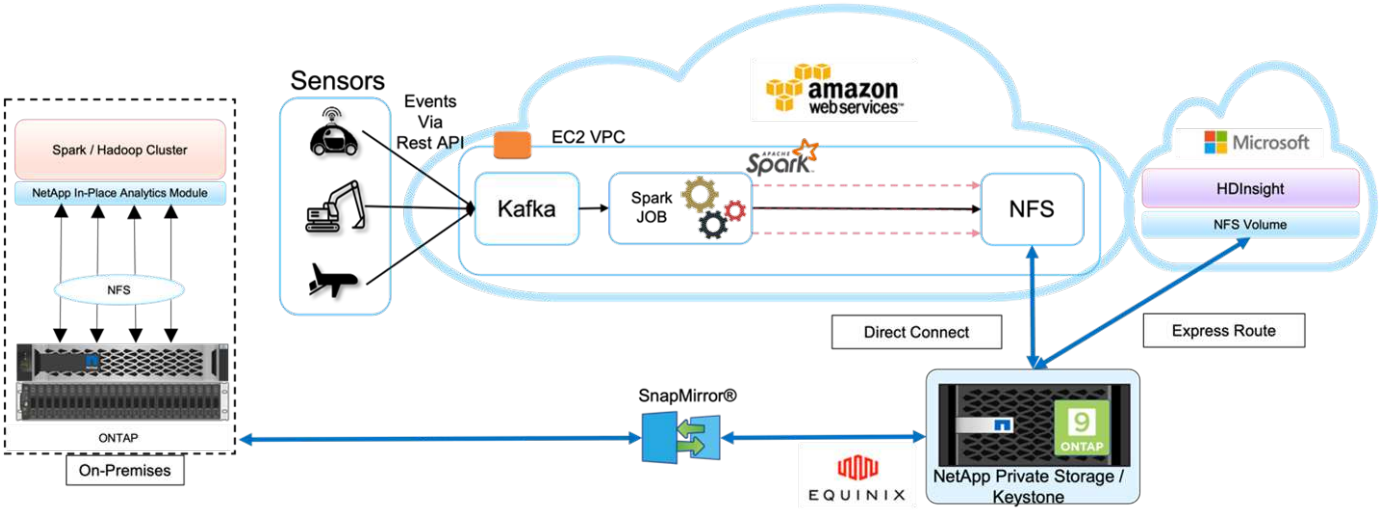
"Anterior: Resultados de las pruebas."

Un centro de datos empresarial moderno es un cloud híbrido que conecta varios entornos de infraestructura distribuidos a través de un plano de gestión de datos continua con un modelo operativo consistente, en las instalaciones o en varios clouds públicos. Para sacarle el máximo partido a un cloud híbrido, debe ser capaz de mover datos con fluidez entre sus entornos locales y varios cloud sin necesidad de realizar conversiones de datos o refactorizaciones de aplicaciones.

Los clientes han indicado que inician su transición al cloud híbrido mediante el movimiento de almacenamiento secundario al cloud para casos de uso como la protección de datos o trasladando cargas de trabajo menos vitales para el negocio como el desarrollo de aplicaciones y DevOps al cloud. Después, se mueven a cargas de trabajo más críticas. El alojamiento web y de contenido, DevOps y el desarrollo de aplicaciones, bases de datos, análisis y aplicaciones en contenedores son algunas de las cargas de trabajo de cloud híbrido más populares. La complejidad, el coste y los riesgos de los proyectos de IA empresarial han dificultado históricamente la adopción de la IA desde la fase experimental hasta la producción.

Con una solución de cloud híbrido de NetApp, los clientes se benefician de las herramientas integradas de seguridad, gobernanza de datos y cumplimiento de normativas, con un único panel de control para la gestión de datos y flujos de trabajo en entornos distribuidos, al tiempo que optimizan el coste total de propiedad en función de su consumo. La siguiente figura representa una solución de ejemplo de un partner de servicio de

cloud que ha encargado de proporcionar conectividad multicloud para los datos de análisis de Big Data de los clientes.



En este escenario, los datos de IoT recibidos en AWS de diferentes orígenes se almacenan en una ubicación central en el almacenamiento privado de NetApp (NPS). El almacenamiento NPS está conectado a clústeres de Spark o Hadoop ubicados en AWS y Azure y permite que se ejecuten aplicaciones de análisis de Big Data en varios clouds accediendo a los mismos datos. Entre los principales requisitos y retos para este caso de uso se encuentran los siguientes:

- Los clientes quieren ejecutar trabajos de análisis en los mismos datos utilizando varios clouds.
- Los datos deben recibirse de diferentes fuentes, como los entornos locales y de cloud, mediante distintos sensores y concentradores.
- La solución debe ser eficiente y rentable.
- El principal reto consiste en crear una solución rentable y eficiente que ofrezca servicios de análisis híbridos entre diferentes entornos de cloud y locales.

Nuestra solución de conectividad multicloud y protección de datos resuelve el problema de tener aplicaciones de análisis en cloud en múltiples proveedores a hiperescala. Como se muestra en la figura anterior, los datos de los sensores se transmiten y ingieren en el clúster de AWS Spark a través de Kafka. Los datos se almacenan en un recurso compartido de NFS que reside en NPS, que se encuentra fuera del proveedor de cloud en un centro de datos Equinix.

Dado que el almacenamiento privado de NetApp se conecta con Amazon AWS y Microsoft Azure mediante las conexiones Direct Connect y Express Route respectivamente, los clientes pueden aprovechar el módulo de análisis in situ para acceder a los datos desde los clústeres de análisis de Amazon y AWS. Por lo tanto, dado que tanto el almacenamiento local como el NPS ejecutan el software ONTAP, "SnapMirror" Puede reflejar los datos de NPS en el clúster en las instalaciones, lo que proporciona análisis en el cloud híbrido entre las instalaciones y varios clouds.

Para obtener el mejor rendimiento, NetApp suele recomendar el uso de varias interfaces de red, así como una conexión directa o rutas exprés para acceder a los datos desde instancias del cloud. Tenemos otras soluciones de movimiento de datos, incluyendo "XCP" y.. "Cloud Sync" Para ayudar a los clientes a crear clústeres de Spark de cloud híbrido rentables, seguros y compatibles con aplicaciones.

"Siguiente: Scripts de Python para cada caso de uso principal."

Scripts Python para cada caso de uso principal

"Anterior: Solución de cloud híbrido."

Las tres secuencias de comandos de Python siguientes corresponden a los tres casos de uso principales probados. La primera es `sentiment_analysis_sparknlp.py`.

```
# TR-4570 Refresh NLP testing by Rick Huang
from sys import argv
import os
import sparknlp
import pyspark.sql.functions as F
from sparknlp import Finisher
from pyspark.ml import Pipeline
from sparknlp.base import *
from sparknlp.annotator import *
from sparknlp.pretrained import PretrainedPipeline
from sparknlp import Finisher
# Start Spark Session with Spark NLP
spark = sparknlp.start()
print("Spark NLP version:")
print(sparknlp.version())
print("Apache Spark version:")
print(spark.version)
spark = sparknlp.SparkSession.builder \
     .master("yarn") \
     .appName("test_hdfs_read_write") \
     .config("spark.executor.cores", "1") \
     .config("spark.jars.packages", "com.johnsnowlabs.nlp:spark-
nlp_2.12:3.4.3")\
     .config('spark.executor.memory', '5gb') \
     .config('spark.executor.memoryOverhead','1000')\
     .config('spark.driver.memoryOverhead','1000')\
     .config("spark.sql.shuffle.partitions", "480")\
     .getOrCreate()
sc = spark.sparkContext
from pyspark.sql import SQLContext
sql = SQLContext(sc)
sqlContext = SQLContext(sc)
# Download pre-trained pipelines & sequence classifier
explain_pipeline_model = PretrainedPipeline('explain_document_dl',
lang='en').model#pipeline_sa =
PretrainedPipeline("classifierdl_bertwiki_finance_sentiment_pipeline",
lang="en")
# pipeline_finbert =
BertForSequenceClassification.loadSavedModel('/sparkusecase/bert_sequence_
```

```
classifier_finbert_en_3', spark)
sequenceClassifier = BertForSequenceClassification \
        .pretrained('bert_sequence_classifier_finbert', 'en') \
        .setInputCols(['token', 'document']) \
        .setOutputCol('class') \
        .setCaseSensitive(True) \
        .setMaxSentenceLength(512)
def process_sentence_df(data):
    # Pre-process: begin
    print("1. Begin DataFrame pre-processing...\n")
    print(f"\n\t2. Attaching DocumentAssembler Transformer to the
pipeline")
    documentAssembler = DocumentAssembler() \
        .setInputCol("text") \
        .setOutputCol("document") \
        .setCleanupMode("inplace_full")
        #.setCleanupMode("shrink", "inplace_full")
    doc_df = documentAssembler.transform(data)
    doc_df.printSchema()
    doc_df.show(truncate=50)
    # Pre-process: get rid of  blank lines
    clean_df = doc_df.withColumn("tmp", F.explode("document")) \
        .select("tmp.result").where("tmp.end !=
-1").withColumnRenamed("result", "text").dropna()
    print("[OK!] DataFrame after initial cleanup:\n")
    clean_df.printSchema()
    clean_df.show(truncate=80)
    # for FinBERT
    tokenizer = Tokenizer() \
        .setInputCols(['document']) \
        .setOutputCol('token')
    print(f"\n\t3. Attaching Tokenizer Annotator to the pipeline")
    pipeline_finbert = Pipeline(stages=[
        documentAssembler,
        tokenizer,
        sequenceClassifier
        ])
    # Use Finisher() & construct PySpark ML pipeline
    finisher = Finisher().setInputCols(["token", "lemma", "pos",
"entities"])
    print(f"\n\t4. Attaching Finisher Transformer to the pipeline")
    pipeline_ex = Pipeline() \
        .setStages([
           explain_pipeline_model,
           finisher
           ])
```

```
    print("\n\t\t\t ---- Pipeline Built Successfully ----")
    # Loading pipelines to annotate
    #result_ex_df = pipeline_ex.transform(clean_df)
    ex_model = pipeline_ex.fit(clean_df)
    annotations_finished_ex_df = ex_model.transform(clean_df)
    # result_sa_df = pipeline_sa.transform(clean_df)
    result_finbert_df = pipeline_finbert.fit(clean_df).transform(clean_df)
    print("\n\t\t\t ----Document Explain, Sentiment Analysis & FinBERT
Pipeline Fitted Successfully ----")
    # Check the result entities
    print("[OK!] Simple explain ML pipeline result:\n")
    annotations_finished_ex_df.printSchema()
    annotations_finished_ex_df.select('text',
'finished_entities').show(truncate=False)
    # Check the result sentiment from FinBERT
    print("[OK!] Sentiment Analysis FinBERT pipeline result:\n")
    result_finbert_df.printSchema()
    result_finbert_df.select('text', 'class.result').show(80, False)
    sentiment_stats(result_finbert_df)
    return
def sentiment_stats(finbert_df):
    result_df = finbert_df.select('text', 'class.result')
    sa_df = result_df.select('result')
    sa_df.groupBy('result').count().show()
    # total_lines = result_clean_df.count()
    # num_neutral = result_clean_df.where(result_clean_df.result ==
['neutral']).count()
    # num_positive = result_clean_df.where(result_clean_df.result ==
['positive']).count()
    # num_negative = result_clean_df.where(result_clean_df.result ==
['negative']).count()
    # print(f"\nRatio of neutral sentiment = {num_neutral/total_lines}")
    # print(f"Ratio of positive sentiment = {num_positive / total_lines}")
    # print(f"Ratio of negative sentiment = {num_negative /
total_lines}\n")
    return
def process_input_file(file_name):
    # Turn input file to Spark DataFrame
    print("START processing input file...")
    data_df = spark.read.text(file_name)
    data_df.show()
    # rename first column 'text' for sparknlp
    output_df = data_df.withColumnRenamed("value", "text").dropna()
    output_df.printSchema()
    return output_dfdef process_local_dir(directory):
    filelist = []
```

```
    for subdir, dirs, files in os.walk(directory):
        for filename in files:
            filepath = subdir + os.sep + filename
            print("[OK!] Will process the following files:")
            if filepath.endswith(".txt"):
                print(filepath)
                filelist.append(filepath)
    return filelist
def process_local_dir_or_file(dir_or_file):
    numfiles = 0
    if os.path.isfile(dir_or_file):
        input_df = process_input_file(dir_or_file)
        print("Obtained input_df.")
        process_sentence_df(input_df)
        print("Processed input_df")
        numfiles += 1
    else:
        filelist = process_local_dir(dir_or_file)
        for file in filelist:
            input_df = process_input_file(file)
            process_sentence_df(input_df)
            numfiles += 1
    return numfiles
def process_hdfs_dir(dir_name):
    # Turn input files to Spark DataFrame
    print("START processing input HDFS directory...")
    data_df = spark.read.option("recursiveFileLookup",
"true").text(dir_name)
    data_df.show()
    print("[DEBUG] total lines in data_df = ", data_df.count())
    # rename first column 'text' for sparknlp
    output_df = data_df.withColumnRenamed("value", "text").dropna()
    print("[DEBUG] output_df looks like: \n")
    output_df.show(40, False)
    print("[DEBUG] HDFS dir resulting data_df schema: \n")
    output_df.printSchema()
    process_sentence_df(output_df)
    print("Processed HDFS directory: ", dir_name)
    returnif __name__ == '__main__':
    try:
        if len(argv) == 2:
            print("Start processing input...\n")
    except:
        print("[ERROR] Please enter input text file or path to
process!\n")
        exit(1)
```

```
    # This is for local file, not hdfs:
    numfiles = process_local_dir_or_file(str(argv[1]))
    # For HDFS single file & directory:
    input_df = process_input_file(str(argv[1]))
    print("Obtained input_df.")
    process_sentence_df(input_df)
    print("Processed input_df")
    numfiles += 1
    # For HDFS directory of subdirectories of files:
    input_parse_list = str(argv[1]).split('/')
    print(input_parse_list)
    if input_parse_list[-2:-1] == ['Transcripts']:
        print("Start processing HDFS directory: ", str(argv[1]))
        process_hdfs_dir(str(argv[1]))
    print(f"[OK!] All done. Number of files processed = {numfiles}")
```

El segundo guión es `keras_spark_horovod_rossmann_estimator.py`.

```
# Copyright 2022 NetApp, Inc.
# Authored by Rick Huang
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
#
==========================================================================
====
# The below code was modified from: https://www.kaggle.com/c/rossmann-
store-sales
import argparse
import datetime
import os
import sys
from distutils.version import LooseVersion
import pyspark.sql.types as T
import pyspark.sql.functions as F
from pyspark import SparkConf, Row
```

```
from pyspark.sql import SparkSession
import tensorflow as tf
import tensorflow.keras.backend as K
from tensorflow.keras.layers import Input, Embedding, Concatenate, Dense,
Flatten, Reshape, BatchNormalization, Dropout
import horovod.spark.keras as hvd
from horovod.spark.common.backend import SparkBackend
from horovod.spark.common.store import Store
from horovod.tensorflow.keras.callbacks import BestModelCheckpoint
parser = argparse.ArgumentParser(description='Horovod Keras Spark Rossmann
Estimator Example',

formatter_class=argparse.ArgumentDefaultsHelpFormatter)
parser.add_argument('--master',
                    help='spark cluster to use for training. If set to
None, uses current default cluster. Cluster'
                         'should be set up to provide a Spark task per
multiple CPU cores, or per GPU, e.g. by'
                         'supplying `-c <NUM_GPUS>` in Spark Standalone
mode')
parser.add_argument('--num-proc', type=int,
                    help='number of worker processes for training,
default: `spark.default.parallelism`')
parser.add_argument('--learning_rate', type=float, default=0.0001,
                    help='initial learning rate')
parser.add_argument('--batch-size', type=int, default=100,
                    help='batch size')
parser.add_argument('--epochs', type=int, default=100,
                    help='number of epochs to train')
parser.add_argument('--sample-rate', type=float,
                    help='desired sampling rate. Useful to set to low
number (e.g. 0.01) to make sure that '
                         'end-to-end process works')
parser.add_argument('--data-dir', default='file://' + os.getcwd(),
                    help='location of data on local filesystem (prefixed
with file://) or on HDFS')
parser.add_argument('--local-submission-csv', default='submission.csv',
                    help='output submission predictions CSV')
parser.add_argument('--local-checkpoint-file', default='checkpoint',
                    help='model checkpoint')
parser.add_argument('--work-dir', default='/tmp',
                    help='temporary working directory to write
intermediate files (prefix with hdfs:// to use HDFS)')
if __name__ == '__main__':
    args = parser.parse_args()
    # ================ #
```

```
    # DATA PREPARATION #
    # ================ #
    print('================')
    print('Data preparation')
    print('================')
    # Create Spark session for data preparation.
    conf = SparkConf() \
        .setAppName('Keras Spark Rossmann Estimator Example') \
        .set('spark.sql.shuffle.partitions', '480') \
        .set("spark.executor.cores", "1") \
        .set('spark.executor.memory', '5gb') \
        .set('spark.executor.memoryOverhead','1000')\
        .set('spark.driver.memoryOverhead','1000')
    if args.master:
        conf.setMaster(args.master)
    elif args.num_proc:
        conf.setMaster('local[{}]'.format(args.num_proc))
    spark = SparkSession.builder.config(conf=conf).getOrCreate()
    train_csv = spark.read.csv('%s/train.csv' % args.data_dir,
header=True)
    test_csv = spark.read.csv('%s/test.csv' % args.data_dir, header=True)
    store_csv = spark.read.csv('%s/store.csv' % args.data_dir,
header=True)
    store_states_csv = spark.read.csv('%s/store_states.csv' %
args.data_dir, header=True)
    state_names_csv = spark.read.csv('%s/state_names.csv' % args.data_dir,
header=True)
    google_trend_csv = spark.read.csv('%s/googletrend.csv' %
args.data_dir, header=True)
    weather_csv = spark.read.csv('%s/weather.csv' % args.data_dir,
header=True)
    def expand_date(df):
        df = df.withColumn('Date', df.Date.cast(T.DateType()))
        return df \
            .withColumn('Year', F.year(df.Date)) \
            .withColumn('Month', F.month(df.Date)) \
            .withColumn('Week', F.weekofyear(df.Date)) \
            .withColumn('Day', F.dayofmonth(df.Date))
    def prepare_google_trend():
        # Extract week start date and state.
        google_trend_all = google_trend_csv \
            .withColumn('Date', F.regexp_extract(google_trend_csv.week,
'(.*?) -', 1)) \
            .withColumn('State', F.regexp_extract(google_trend_csv.file,
'Rossmann_DE_(.*)', 1))
        # Map state NI -> HB,NI to align with other data sources.
```

```
        google_trend_all = google_trend_all \
            .withColumn('State', F.when(google_trend_all.State == 'NI',
'HB,NI').otherwise(google_trend_all.State))
        # Expand dates.
        return expand_date(google_trend_all)
    def add_elapsed(df, cols):
        def add_elapsed_column(col, asc):
            def fn(rows):
                last_store, last_date = None, None
                for r in rows:
                    if last_store != r.Store:
                        last_store = r.Store
                        last_date = r.Date
                    if r[col]:
                        last_date = r.Date
                    fields = r.asDict().copy()
                    fields[('After' if asc else 'Before') + col] = (r.Date
- last_date).days
                    yield Row(**fields)
            return fn
        df = df.repartition(df.Store)
        for asc in [False, True]:
            sort_col = df.Date.asc() if asc else df.Date.desc()
            rdd = df.sortWithinPartitions(df.Store.asc(), sort_col).rdd
            for col in cols:
                rdd = rdd.mapPartitions(add_elapsed_column(col, asc))
            df = rdd.toDF()
        return df
    def prepare_df(df):
        num_rows = df.count()
        # Expand dates.
        df = expand_date(df)
        df = df \
            .withColumn('Open', df.Open != '0') \
            .withColumn('Promo', df.Promo != '0') \
            .withColumn('StateHoliday', df.StateHoliday != '0') \
            .withColumn('SchoolHoliday', df.SchoolHoliday != '0')
        # Merge in store information.
        store = store_csv.join(store_states_csv, 'Store')
        df = df.join(store, 'Store')
        # Merge in Google Trend information.
        google_trend_all = prepare_google_trend()
        df = df.join(google_trend_all, ['State', 'Year',
'Week']).select(df['*'], google_trend_all.trend)
        # Merge in Google Trend for whole Germany.
        google_trend_de = google_trend_all[google_trend_all.file ==
```

```
'Rossmann_DE'].withColumnRenamed('trend', 'trend_de')
        df = df.join(google_trend_de, ['Year', 'Week']).select(df['*'],
google_trend_de.trend_de)
        # Merge in weather.
        weather = weather_csv.join(state_names_csv, weather_csv.file ==
state_names_csv.StateName)
        df = df.join(weather, ['State', 'Date'])
        # Fix null values.
        df = df \
            .withColumn('CompetitionOpenSinceYear',
F.coalesce(df.CompetitionOpenSinceYear, F.lit(1900))) \
            .withColumn('CompetitionOpenSinceMonth',
F.coalesce(df.CompetitionOpenSinceMonth, F.lit(1))) \
            .withColumn('Promo2SinceYear', F.coalesce(df.Promo2SinceYear,
F.lit(1900))) \
            .withColumn('Promo2SinceWeek', F.coalesce(df.Promo2SinceWeek,
F.lit(1)))
        # Days & months competition was open, cap to 2 years.
        df = df.withColumn('CompetitionOpenSince',
                           F.to_date(F.format_string('%s-%s-15',
df.CompetitionOpenSinceYear,

df.CompetitionOpenSinceMonth)))
        df = df.withColumn('CompetitionDaysOpen',
                           F.when(df.CompetitionOpenSinceYear > 1900,
                                  F.greatest(F.lit(0), F.least(F.lit(360 *
2), F.datediff(df.Date, df.CompetitionOpenSince))))
                           .otherwise(0))
        df = df.withColumn('CompetitionMonthsOpen',
(df.CompetitionDaysOpen / 30).cast(T.IntegerType()))
        # Days & weeks of promotion, cap to 25 weeks.
        df = df.withColumn('Promo2Since',
                           F.expr('date_add(format_string("%s-01-01",
Promo2SinceYear), (cast(Promo2SinceWeek as int) - 1) * 7)'))
        df = df.withColumn('Promo2Days',
                           F.when(df.Promo2SinceYear > 1900,
                                  F.greatest(F.lit(0), F.least(F.lit(25 *
7), F.datediff(df.Date, df.Promo2Since))))
                           .otherwise(0))
        df = df.withColumn('Promo2Weeks', (df.Promo2Days /
7).cast(T.IntegerType()))
        # Check that we did not lose any rows through inner joins.
        assert num_rows == df.count(), 'lost rows in joins'
        return df
    def build_vocabulary(df, cols):
        vocab = {}
```

```
        for col in cols:
            values = [r[0] for r in df.select(col).distinct().collect()]
            col_type = type([x for x in values if x is not None][0])
            default_value = col_type()
            vocab[col] = sorted(values, key=lambda x: x or default_value)
        return vocab
    def cast_columns(df, cols):
        for col in cols:
            df = df.withColumn(col,
F.coalesce(df[col].cast(T.FloatType()), F.lit(0.0)))
        return df
    def lookup_columns(df, vocab):
        def lookup(mapping):
            def fn(v):
                return mapping.index(v)
            return F.udf(fn, returnType=T.IntegerType())
        for col, mapping in vocab.items():
            df = df.withColumn(col, lookup(mapping)(df[col]))
        return df
    if args.sample_rate:
        train_csv = train_csv.sample(withReplacement=False,
fraction=args.sample_rate)
        test_csv = test_csv.sample(withReplacement=False,
fraction=args.sample_rate)
    # Prepare data frames from CSV files.
    train_df = prepare_df(train_csv).cache()
    test_df = prepare_df(test_csv).cache()
    # Add elapsed times from holidays & promos, the data spanning training
& test datasets.
    elapsed_cols = ['Promo', 'StateHoliday', 'SchoolHoliday']
    elapsed = add_elapsed(train_df.select('Date', 'Store', *elapsed_cols)
                          .unionAll(test_df.select('Date', 'Store',
*elapsed_cols)),
                          elapsed_cols)
    # Join with elapsed times.
    train_df = train_df \
        .join(elapsed, ['Date', 'Store']) \
        .select(train_df['*'], *[prefix + col for prefix in ['Before',
'After'] for col in elapsed_cols])
    test_df = test_df \
        .join(elapsed, ['Date', 'Store']) \
        .select(test_df['*'], *[prefix + col for prefix in ['Before',
'After'] for col in elapsed_cols])
    # Filter out zero sales.
    train_df = train_df.filter(train_df.Sales > 0)
    print('===================')
```

```
    print('Prepared data frame')
    print('===================')
    train_df.show()
    categorical_cols = [
        'Store', 'State', 'DayOfWeek', 'Year', 'Month', 'Day', 'Week',
'CompetitionMonthsOpen', 'Promo2Weeks', 'StoreType',
        'Assortment', 'PromoInterval', 'CompetitionOpenSinceYear',
'Promo2SinceYear', 'Events', 'Promo',
        'StateHoliday', 'SchoolHoliday'
    ]
    continuous_cols = [
        'CompetitionDistance', 'Max_TemperatureC', 'Mean_TemperatureC',
'Min_TemperatureC', 'Max_Humidity',
        'Mean_Humidity', 'Min_Humidity', 'Max_Wind_SpeedKm_h',
'Mean_Wind_SpeedKm_h', 'CloudCover', 'trend', 'trend_de',
        'BeforePromo', 'AfterPromo', 'AfterStateHoliday',
'BeforeStateHoliday', 'BeforeSchoolHoliday', 'AfterSchoolHoliday'
    ]
    all_cols = categorical_cols + continuous_cols
    # Select features.
    train_df = train_df.select(*(all_cols + ['Sales', 'Date'])).cache()
    test_df = test_df.select(*(all_cols + ['Id', 'Date'])).cache()
    # Build vocabulary of categorical columns.
    vocab = build_vocabulary(train_df.select(*categorical_cols)

.unionAll(test_df.select(*categorical_cols)).cache(),
                              categorical_cols)
    # Cast continuous columns to float & lookup categorical columns.
    train_df = cast_columns(train_df, continuous_cols + ['Sales'])
    train_df = lookup_columns(train_df, vocab)
    test_df = cast_columns(test_df, continuous_cols)
    test_df = lookup_columns(test_df, vocab)
    # Split into training & validation.
    # Test set is in 2015, use the same period in 2014 from the training
set as a validation set.
    test_min_date = test_df.agg(F.min(test_df.Date)).collect()[0][0]
    test_max_date = test_df.agg(F.max(test_df.Date)).collect()[0][0]
    one_year = datetime.timedelta(365)
    train_df = train_df.withColumn('Validation',
                                   (train_df.Date > test_min_date -
one_year) & (train_df.Date <= test_max_date - one_year))
    # Determine max Sales number.
    max_sales = train_df.agg(F.max(train_df.Sales)).collect()[0][0]
    # Convert Sales to log domain
    train_df = train_df.withColumn('Sales', F.log(train_df.Sales))
    print('====================================')
```

```
    print('Data frame with transformed columns')
    print('===================================')
    train_df.show()
    print('================')
    print('Data frame sizes')
    print('================')
    train_rows = train_df.filter(~train_df.Validation).count()
    val_rows = train_df.filter(train_df.Validation).count()
    test_rows = test_df.count()
    print('Training: %d' % train_rows)
    print('Validation: %d' % val_rows)
    print('Test: %d' % test_rows)
    # ============== #
    # MODEL TRAINING #
    # ============== #
    print('==============')
    print('Model training')
    print('==============')
    def exp_rmspe(y_true, y_pred):
        """Competition evaluation metric, expects logarithic inputs."""
        pct = tf.square((tf.exp(y_true) - tf.exp(y_pred)) /
tf.exp(y_true))
        # Compute mean excluding stores with zero denominator.
        x = tf.reduce_sum(tf.where(y_true > 0.001, pct,
tf.zeros_like(pct)))
        y = tf.reduce_sum(tf.where(y_true > 0.001, tf.ones_like(pct),
tf.zeros_like(pct)))
        return tf.sqrt(x / y)
    def act_sigmoid_scaled(x):
        """Sigmoid scaled to logarithm of maximum sales scaled by 20%."""
        return tf.nn.sigmoid(x) * tf.math.log(max_sales) * 1.2
    CUSTOM_OBJECTS = {'exp_rmspe': exp_rmspe,
                      'act_sigmoid_scaled': act_sigmoid_scaled}
    # Disable GPUs when building the model to prevent memory leaks
    if LooseVersion(tf.__version__) >= LooseVersion('2.0.0'):
        # See https://github.com/tensorflow/tensorflow/issues/33168
        os.environ['CUDA_VISIBLE_DEVICES'] = '-1'
    else:

K.set_session(tf.Session(config=tf.ConfigProto(device_count={'GPU': 0})))
    # Build the model.
    inputs = {col: Input(shape=(1,), name=col) for col in all_cols}
    embeddings = [Embedding(len(vocab[col]), 10, input_length=1,
name='emb_' + col)(inputs[col])
                  for col in categorical_cols]
    continuous_bn = Concatenate()([Reshape((1, 1), name='reshape_' +
```

```
col)(inputs[col])
                                    for col in continuous_cols])
    continuous_bn = BatchNormalization()(continuous_bn)
    x = Concatenate()(embeddings + [continuous_bn])
    x = Flatten()(x)
    x = Dense(1000, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.00005))(x)
    x = Dense(1000, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.00005))(x)
    x = Dense(1000, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.00005))(x)
    x = Dense(500, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.00005))(x)
    x = Dropout(0.5)(x)
    output = Dense(1, activation=act_sigmoid_scaled)(x)
    model = tf.keras.Model([inputs[f] for f in all_cols], output)
    model.summary()
    opt = tf.keras.optimizers.Adam(lr=args.learning_rate, epsilon=1e-3)
    # Checkpoint callback to specify options for the returned Keras model
    ckpt_callback = BestModelCheckpoint(monitor='val_loss', mode='auto',
save_freq='epoch')
    # Horovod: run training.
    store = Store.create(args.work_dir)
    backend = SparkBackend(num_proc=args.num_proc,
                           stdout=sys.stdout, stderr=sys.stderr,
                           prefix_output_with_timestamp=True)
    keras_estimator = hvd.KerasEstimator(backend=backend,
                                         store=store,
                                         model=model,
                                         optimizer=opt,
                                         loss='mae',
                                         metrics=[exp_rmspe],
                                         custom_objects=CUSTOM_OBJECTS,
                                         feature_cols=all_cols,
                                         label_cols=['Sales'],
                                         validation='Validation',
                                         batch_size=args.batch_size,
                                         epochs=args.epochs,
                                         verbose=2,

checkpoint_callback=ckpt_callback)
    keras_model =
keras_estimator.fit(train_df).setOutputCols(['Sales_output'])
    history = keras_model.getHistory()
    best_val_rmspe = min(history['val_exp_rmspe'])
    print('Best RMSPE: %f' % best_val_rmspe)
```

```
    # Save the trained model.
    keras_model.save(args.local_checkpoint_file)
    print('Written checkpoint to %s' % args.local_checkpoint_file)
    # ================ #
    # FINAL PREDICTION #
    # ================ #
    print('================')
    print('Final prediction')
    print('================')
    pred_df=keras_model.transform(test_df)
    pred_df.printSchema()
    pred_df.show(5)
    # Convert from log domain to real Sales numbers
    pred_df=pred_df.withColumn('Sales_pred', F.exp(pred_df.Sales_output))
    submission_df = pred_df.select(pred_df.Id.cast(T.IntegerType()),
pred_df.Sales_pred).toPandas()
    submission_df.sort_values(by=['Id']).to_csv(args.local_submission_csv,
index=False)
    print('Saved predictions to %s' % args.local_submission_csv)
    spark.stop()
```

El tercer guión es `run_classification_criteo_spark.py`.

```
import tempfile, string, random, os, uuid
import argparse, datetime, sys, shutil
import csv
import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow.keras.callbacks import EarlyStopping
from pyspark import SparkContext
from pyspark.sql import SparkSession, SQLContext, Row, DataFrame
from pyspark.mllib import linalg as mllib_linalg
from pyspark.mllib.linalg import SparseVector as mllibSparseVector
from pyspark.mllib.linalg import VectorUDT as mllibVectorUDT
from pyspark.mllib.linalg import Vector as mllibVector, Vectors as
mllibVectors
from pyspark.mllib.regression import LabeledPoint
from pyspark.mllib.classification import LogisticRegressionWithSGD
from pyspark.ml import linalg as ml_linalg
from pyspark.ml.linalg import VectorUDT as mlVectorUDT
from pyspark.ml.linalg import SparseVector as mlSparseVector
from pyspark.ml.linalg import Vector as mlVector, Vectors as mlVectors
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.feature import OneHotEncoder
from math import log
```

```
from math import exp  # exp(-t) = e^-t
from operator import add
from pyspark.sql.functions import udf, split, lit
from pyspark.sql.functions import size, sum as sqlsum
import pyspark.sql.functions as F
import pyspark.sql.types as T
from pyspark.sql.types import ArrayType, StructType, StructField,
LongType, StringType, IntegerType, FloatType
from pyspark.sql.functions import explode, col, log, when
from collections import defaultdict
import pandas as pd
import pyspark.pandas as ps
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from deepctr.models import DeepFM
from deepctr.feature_column import SparseFeat, DenseFeat,
get_feature_names
spark = SparkSession.builder \
    .master("yarn") \
    .appName("deep_ctr_classification") \
    .config("spark.jars.packages", "io.github.ravwojdyla:spark-schema-
utils_2.12:0.1.0") \
    .config("spark.executor.cores", "1") \
    .config('spark.executor.memory', '5gb') \
    .config('spark.executor.memoryOverhead', '1500') \
    .config('spark.driver.memoryOverhead', '1500') \
    .config("spark.sql.shuffle.partitions", "480") \
    .config("spark.sql.execution.arrow.enabled", "true") \
    .config("spark.driver.maxResultSize", "50gb") \
    .getOrCreate()
# spark.conf.set("spark.sql.execution.arrow.enabled", "true") # deprecated
print("Apache Spark version:")
print(spark.version)
sc = spark.sparkContext
sqlContext = SQLContext(sc)
parser = argparse.ArgumentParser(description='Spark DCN CTR Prediction
Example',

formatter_class=argparse.ArgumentDefaultsHelpFormatter)
parser.add_argument('--data-dir', default='file://' + os.getcwd(),
                    help='location of data on local filesystem (prefixed
with file://) or on HDFS')
def process_input_file(file_name, sparse_feat, dense_feat):
    # Need this preprocessing to turn Criteo raw file into CSV:
    print("START processing input file...")
```

```
    # only convert the file ONCE
    # sample = open(file_name)
    # sample = '\n'.join([str(x.replace('\n', '').replace('\t', ',')) for
x in sample])
    # # Add header in data file and save as CSV
    # header = ','.join(str(x) for x in (['label'] + dense_feat +
sparse_feat))
    # with open('/sparkdemo/tr-4570-data/ctr_train.csv', mode='w',
encoding="utf-8") as f:
    #     f.write(header + '\n' + sample)
    #     f.close()
    # print("Raw training file processed and saved as CSV: ", f.name)
    raw_df = sqlContext.read.option("header", True).csv(file_name)
    raw_df.show(5, False)
    raw_df.printSchema()
    # convert columns I1 to I13 from string to integers
    conv_df = raw_df.select(col('label').cast("double"),
                            *(col(i).cast("float").alias(i) for i in
raw_df.columns if i in dense_feat),
                            *(col(c) for c in raw_df.columns if c in
sparse_feat))
    print("Schema of raw_df with integer columns type changed:")
    conv_df.printSchema()
    # result_pdf = conv_df.select("*").toPandas()
    tmp_df = conv_df.na.fill(0, dense_feat)
    result_df = tmp_df.na.fill('-1', sparse_feat)
    result_df.show()
    return result_df
if __name__ == "__main__":
    args = parser.parse_args()
    # Pandas read CSV
    # data = pd.read_csv('%s/criteo_sample.txt' % args.data_dir)
    # print("Obtained Pandas df.")
    dense_features = ['I' + str(i) for i in range(1, 14)]
    sparse_features = ['C' + str(i) for i in range(1, 27)]
    # Spark read CSV
    # process_input_file('%s/train.txt' % args.data_dir, sparse_features,
dense_features) # run only ONCE
    spark_df = process_input_file('%s/data.txt' % args.data_dir,
sparse_features, dense_features) # sample data
    # spark_df = process_input_file('%s/ctr_train.csv' % args.data_dir,
sparse_features, dense_features)
    print("Obtained Spark df and filled in missing features.")
    data = spark_df
    # Pandas
    #data[sparse_features] = data[sparse_features].fillna('-1', )
```

```
    #data[dense_features] = data[dense_features].fillna(0, )
    target = ['label']
    label_npa = data.select("label").toPandas().to_numpy()
    print("label numPy array has length = ", len(label_npa)) # 45,840,617
w/ 11GB dataset
    label_npa.ravel()
    label_npa.reshape(len(label_npa), )
    # 1.Label Encoding for sparse features,and do simple Transformation
for dense features
    print("Before LabelEncoder():")
    data.printSchema()  # label: float (nullable = true)
    for feat in sparse_features:
        lbe = LabelEncoder()
        tmp_pdf = data.select(feat).toPandas().to_numpy()
        tmp_ndarray = lbe.fit_transform(tmp_pdf)
        print("After LabelEncoder(), tmp_ndarray[0] =", tmp_ndarray[0])
        # print("Data tmp PDF after lbe transformation, the output ndarray
has length = ", len(tmp_ndarray)) # 45,840,617 for 11GB dataset
        tmp_ndarray.ravel()
        tmp_ndarray.reshape(len(tmp_ndarray), )
        out_ndarray = np.column_stack([label_npa, tmp_ndarray])
        pdf = pd.DataFrame(out_ndarray, columns=['label', feat])
        s_df = spark.createDataFrame(pdf)
        s_df.printSchema() # label: double (nullable = true)
        print("Before joining data df with s_df, s_df example rows:")
        s_df.show(1, False)
        data = data.drop(feat).join(s_df, 'label').drop('label')
        print("After LabelEncoder(), data df example rows:")
        data.show(1, False)
        print("Finished processing sparse_features: ", feat)
    print("Data DF after label encoding: ")
    data.show()
    data.printSchema()
    mms = MinMaxScaler(feature_range=(0, 1))
    # data[dense_features] = mms.fit_transform(data[dense_features]) # for
Pandas df
    tmp_pdf = data.select(dense_features).toPandas().to_numpy()
    tmp_ndarray = mms.fit_transform(tmp_pdf)
    tmp_ndarray.ravel()
    tmp_ndarray.reshape(len(tmp_ndarray), len(tmp_ndarray[0]))
    out_ndarray = np.column_stack([label_npa, tmp_ndarray])
    pdf = pd.DataFrame(out_ndarray, columns=['label'] + dense_features)
    s_df = spark.createDataFrame(pdf)
    s_df.printSchema()
    data.drop(*dense_features).join(s_df, 'label').drop('label')
    print("Finished processing dense_features: ", dense_features)
```

```
    print("Data DF after MinMaxScaler: ")
    data.show()

    # 2.count #unique features for each sparse field,and record dense
feature field name
    fixlen_feature_columns = [SparseFeat(feat,
vocabulary_size=data.select(feat).distinct().count() + 1, embedding_dim=4)
                              for i, feat in enumerate(sparse_features)] +
\
                             [DenseFeat(feat, 1, ) for feat in
dense_features]
    dnn_feature_columns = fixlen_feature_columns
    linear_feature_columns = fixlen_feature_columns
    feature_names = get_feature_names(linear_feature_columns +
dnn_feature_columns)
    # 3.generate input data for model
    # train, test = train_test_split(data.toPandas(), test_size=0.2,
random_state=2020) # Pandas; might hang for 11GB data
    train, test = data.randomSplit(weights=[0.8, 0.2], seed=200)
    print("Training dataset size = ", train.count())
    print("Testing dataset size = ", test.count())
    # Pandas:
    # train_model_input = {name: train[name] for name in feature_names}
    # test_model_input = {name: test[name] for name in feature_names}
    # Spark DF:
    train_model_input = {}
    test_model_input = {}
    for name in feature_names:
        if name.startswith('I'):
            tr_pdf = train.select(name).toPandas()
            train_model_input[name] = pd.to_numeric(tr_pdf[name])
            ts_pdf = test.select(name).toPandas()
            test_model_input[name] = pd.to_numeric(ts_pdf[name])
    # 4.Define Model,train,predict and evaluate
    model = DeepFM(linear_feature_columns, dnn_feature_columns,
task='binary')
    model.compile("adam", "binary_crossentropy",
                  metrics=['binary_crossentropy'], )
    lb_pdf = train.select(target).toPandas()
    history = model.fit(train_model_input,
pd.to_numeric(lb_pdf['label']).values,
                        batch_size=256, epochs=10, verbose=2,
validation_split=0.2, )
    pred_ans = model.predict(test_model_input, batch_size=256)
    print("test LogLoss",
round(log_loss(pd.to_numeric(test.select(target).toPandas()).values,
```

```
pred_ans), 4))
    print("test AUC",
round(roc_auc_score(pd.to_numeric(test.select(target).toPandas()).values,
pred_ans), 4))
```

"Siguiente: Conclusión."

Conclusión

"Anterior: Scripts de Python para cada caso de uso principal."

En este documento, comentamos la arquitectura Apache Spark, los casos de uso de clientes y la cartera de almacenamiento de NetApp en relación a Big Data, análisis modernos e IA, ML y DL. En nuestras pruebas de validación del rendimiento basadas en herramientas de pruebas de rendimiento estándares del sector y la demanda de los clientes, las soluciones de Spark de NetApp demostraron un rendimiento superior en comparación con los sistemas Hadoop nativos. Una combinación de los casos de uso de clientes y los resultados de rendimiento que se presentan en este informe puede ayudarle a elegir la solución Spark adecuada para su puesta en marcha.

"Siguiente: Dónde encontrar información adicional."

Dónde encontrar información adicional

"Anterior: Conclusión."

Las siguientes referencias se han utilizado en este informe técnico:

- Arquitectura y componentes de Apache Spark

  "http://spark.apache.org/docs/latest/cluster-overview.html"

- Casos de uso de Apache Spark

  "https://www.qubole.com/blog/big-data/apache-spark-use-cases/"

- Problemas de Apache

  "http://www.infoworld.com/article/2897287/big-data/5-reasons-to-turn-to-spark-for-big-data-analytics.html"

- Chispa NLP

  "https://www.johnsnowlabs.com/spark-nlp/"

- BERT

  "https://arxiv.org/abs/1810.04805"

- Red profunda y cruzada para predicciones de clic de anuncios

  "https://arxiv.org/abs/1708.05123"

- FlexGroup

"http://www.netapp.com/us/media/tr-4557.pdf"

- Streaming ETL

  "https://www.infoq.com/articles/apache-spark-streaming"

- Soluciones E-Series de NetApp para Hadoop

  "https://www.netapp.com/media/16420-tr-3969.pdf"

- Análisis de confianza en las comunicaciones con el cliente con IA de NetApp

  "https://docs.netapp.com/us-en/netapp-solutions/pdfs/sidebar/Sentiment_analysis_with_NetApp_AI.pdf"

- Soluciones de análisis de datos modernas de NetApp

  "https://docs.netapp.com/us-en/netapp-solutions/data-analytics/index.html"

- SnapMirror

  "https://docs.netapp.com/us-en/ontap/data-protection/snapmirror-replication-concept.html"

- XCP

  https://mysupport.netapp.com/documentation/docweb/index.html?productID=63942&language=en-US

- Cloud Sync

  "https://cloud.netapp.com/cloud-sync-service"

- Kit de herramientas de operaciones de datos

  "https://github.com/NetApp/netapp-dataops-toolkit"

Información de copyright

Copyright © 2023 NetApp, Inc. Todos los derechos reservados. Imprimido en EE. UU. No se puede reproducir este documento protegido por copyright ni parte del mismo de ninguna forma ni por ningún medio (gráfico, electrónico o mecánico, incluidas fotocopias, grabaciones o almacenamiento en un sistema de recuperación electrónico) sin la autorización previa y por escrito del propietario del copyright.

El software derivado del material de NetApp con copyright está sujeto a la siguiente licencia y exención de responsabilidad:

ESTE SOFTWARE LO PROPORCIONA NETAPP «TAL CUAL» Y SIN NINGUNA GARANTÍA EXPRESA O IMPLÍCITA, INCLUYENDO, SIN LIMITAR, LAS GARANTÍAS IMPLÍCITAS DE COMERCIALIZACIÓN O IDONEIDAD PARA UN FIN CONCRETO, CUYA RESPONSABILIDAD QUEDA EXIMIDA POR EL PRESENTE DOCUMENTO. EN NINGÚN CASO NETAPP SERÁ RESPONSABLE DE NINGÚN DAÑO DIRECTO, INDIRECTO, ESPECIAL, EJEMPLAR O RESULTANTE (INCLUYENDO, ENTRE OTROS, LA OBTENCIÓN DE BIENES O SERVICIOS SUSTITUTIVOS, PÉRDIDA DE USO, DE DATOS O DE BENEFICIOS, O INTERRUPCIÓN DE LA ACTIVIDAD EMPRESARIAL) CUALQUIERA SEA EL MODO EN EL QUE SE PRODUJERON Y LA TEORÍA DE RESPONSABILIDAD QUE SE APLIQUE, YA SEA EN CONTRATO, RESPONSABILIDAD OBJETIVA O AGRAVIO (INCLUIDA LA NEGLIGENCIA U OTRO TIPO), QUE SURJAN DE ALGÚN MODO DEL USO DE ESTE SOFTWARE, INCLUSO SI HUBIEREN SIDO ADVERTIDOS DE LA POSIBILIDAD DE TALES DAÑOS.

NetApp se reserva el derecho de modificar cualquiera de los productos aquí descritos en cualquier momento y sin aviso previo. NetApp no asume ningún tipo de responsabilidad que surja del uso de los productos aquí descritos, excepto aquello expresamente acordado por escrito por parte de NetApp. El uso o adquisición de este producto no lleva implícita ninguna licencia con derechos de patente, de marcas comerciales o cualquier otro derecho de propiedad intelectual de NetApp.

Es posible que el producto que se describe en este manual esté protegido por una o más patentes de EE. UU., patentes extranjeras o solicitudes pendientes.

LEYENDA DE DERECHOS LIMITADOS: el uso, la copia o la divulgación por parte del gobierno están sujetos a las restricciones establecidas en el subpárrafo (b)(3) de los derechos de datos técnicos y productos no comerciales de DFARS 252.227-7013 (FEB de 2014) y FAR 52.227-19 (DIC de 2007).

Los datos aquí contenidos pertenecen a un producto comercial o servicio comercial (como se define en FAR 2.101) y son propiedad de NetApp, Inc. Todos los datos técnicos y el software informático de NetApp que se proporcionan en este Acuerdo tienen una naturaleza comercial y se han desarrollado exclusivamente con fondos privados. El Gobierno de EE. UU. tiene una licencia limitada, irrevocable, no exclusiva, no transferible, no sublicenciable y de alcance mundial para utilizar los Datos en relación con el contrato del Gobierno de los Estados Unidos bajo el cual se proporcionaron los Datos. Excepto que aquí se disponga lo contrario, los Datos no se pueden utilizar, desvelar, reproducir, modificar, interpretar o mostrar sin la previa aprobación por escrito de NetApp, Inc. Los derechos de licencia del Gobierno de los Estados Unidos de América y su Departamento de Defensa se limitan a los derechos identificados en la cláusula 252.227-7015(b) de la sección DFARS (FEB de 2014).

Información de la marca comercial

NETAPP, el logotipo de NETAPP y las marcas que constan en http://www.netapp.com/TM son marcas comerciales de NetApp, Inc. El resto de nombres de empresa y de producto pueden ser marcas comerciales de sus respectivos propietarios.